Lecture 1 — Introduction & Version Control

> **Lecture id:** L01 **Source decks:** `IntroductionSB5MAI.pdf` (29p) **Labs:** `Lab - GIT.pdf` (19p), `IntroLab.pdf` (1p), `[Litt] Literature List.pdf` (2p, skimmed for citation keys only) **Process phase(s):** Course introduction · Version Control **Citation key:** slides cited as `(Introduction p.X)`, `(Git Lab p.X)`, `(IntroLab p.1)`; readings as `[Raj13]`, `[GHJV94]`, `[MC09]`, etc. (full bibliography in `[Litt] Literature List.pdf`). **Grounding note:** The lecture sets up the *whole* course but does NOT itself walk through the canonical change process (Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion → Verification). That process is introduced in later lectures; here it is only implied by the portfolio brief ("refactor JHotDraw according to Clean Code"). Brooks's "No Silver Bullet" essential/accidental distinction underlies slides 9–10 but is never named on the slides that present those properties — note, however, that **Fred Brooks (1982) *is* named on the Software Engineering History slide** (Introduction p.11), there in connection with **IBM OS/360** and the rise of waterfall/life-span models, not with "No Silver Bullet" — so treat the "No Silver Bullet" attribution as background, not as something the deck states. The Git Lab slides are hand-drawn and image-heavy; command details below were read from rendered slide images, not extractable text.

Overview

Lecture 1 of SB5-MAI ("Software Maintenance", 5 ECTS, University of Southern Denmark, taught by Jan Corfixen Sørensen) is the course **Initiation** lecture. It does two things. First, the **Introduction deck** frames *why software maintenance matters*: software is everywhere ("Internet in everything"), it is hard because of five **essential difficulties** (complexity, invisibility, changeability, conformity, discontinuity), and the history of software engineering has produced three co-existing **paradigms** (ad hoc, waterfall, iterative/agile) and several **software life-span models** (staged, versioned-staged, V-model, prototype) (Introduction p.2, p.8–9, p.27–28). Second, the **Git Lab** plus **IntroLab** bootstrap the practical toolchain: version control with Git/GitHub and the Maven build, applied to the recurring **JHotDraw** case study that every later lecture revisits (Git Lab p.18; IntroLab p.1).

The course's assessment is an **individual on-campus written report** built from a **portfolio**: students refactor JHotDraw features according to Clean Code, graded on the Danish 7-point scale (Introduction p.5–6). So from day one the course connects three threads that recur all semester: a real legacy codebase (JHotDraw), a disciplined change workflow (later formalised as the canonical change process), and version control as the substrate that makes incremental, traceable change possible. This guide is exhaustive over both decks; because the slides are sparse on text and rich in diagrams, the diagram content has been transcribed in full.

Read this lecture as the *map of the whole module*. Nearly every concept introduced here is a "header" for material expanded in later lectures: the essential difficulties motivate every technique that follows; the staged/reengineering model is the economic argument for refactoring; the agile loop is the ancestor of the canonical change process; and the Git four-area model is the mechanical substrate on which all portfolio work is recorded and graded. If you understand *why* each idea is on the slide and *what problem in the change process it addresses*, the rest of the course reads as a series of answers to questions posed here.

This lecture maps to the controlled topics **Software Processes / CI** (paradigms, life-span models), **Version Control / Git**, and **JHotDraw Case Study**, with forward links to **Software Change Process**,

Refactoring, and **Clean Code**.

Learning Objectives

By the end of Lecture 1 a student should be able to:

1. State the **course logistics** of SB5-MAI: 5 ECTS, prerequisites (advanced OO programming ≈12.5 ECTS, software engineering ≈20 ECTS, team work), learning outcome "maintenance of larger existing software projects", and the individual portfolio-based written exam on the 7-point scale (Introduction p.4–6).
2. Explain *why* maintenance dominates modern software: software is pervasive ("software is everywhere") and long-lived (Introduction p.8–9).
3. List and explain the **five essential difficulties** of software — complexity, invisibility, changeability, conformity, discontinuity — and distinguish them from **accidental** properties such as the choice of technology/platform (Introduction p.9–10, p.27).
4. Describe the **three paradigms** (ad hoc, waterfall, iterative/agile), why waterfall fails (requirements volatility, CHAOS-report failure rates), and why the paradigms *co-exist* rather than one replacing another (Introduction p.13–19, p.27).
5. Compare the **software life-span models** — staged, reengineering, versioned-staged, V-model, prototype — and locate maintenance/servicing within them (Introduction p.20–25, p.28).
6. Explain the purpose of **version control** (discipline, history, collaboration, recovery) and the difference between **centralized** and **decentralized** VCS (Git Lab p.3, p.7, p.9).
7. Execute the core **Git workflow** across workspace → index/stage → local repository (HEAD) → remote repository using clone, add, commit, push, fetch, merge, pull, diff (Git Lab p.11).
8. Bootstrap the **JHotDraw** case study: fork on GitHub, build with Maven (JDK 11), run the SVG sample GUI, and follow GitHub-flow feature branches (IntroLab p.1; Git Lab p.18–19).

Key Concepts

Deck roadmap: how the Introduction deck is organised

What it is. The Outline slide (Introduction p.2) declares the deck's structure as four named parts plus the course block: **SB5-MAI: Software Maintenance** (course logistics), **Introduction** (motivation and the essential/accidental analysis), **Paradigms** (ad hoc, waterfall, agile), **Software Life-Span Models** (staged, reengineering, versioned-staged, V-model, prototype), and **Summary**. Each part opens with a full-page section-divider slide: "SB5-MAI: Software Maintenance" (Introduction p.3), "Introduction" (Introduction p.7), "Paradigms" (Introduction p.12), "Software Life-Span Models" (Introduction p.19 region), and "Summary" (Introduction p.26 region). The Motivation slide is a two-step incremental build — first only "Internet in everything", then "Software is everywhere" is added — which is why the PDF contains 29 physical pages for 28 numbered slides (Introduction p.8–9).

What it's used for / why it matters. The outline is itself the deck's taxonomy of the lecture: after the course block, there are exactly **two conceptual pillars** — *paradigms* (philosophies of *how* software is developed) and *life-span models* (descriptions of *what stage* a product is in across its whole existence) — bracketed by motivation and a summary. Recognising this two-pillar structure prevents a common exam confusion: a paradigm and a life-span model answer different questions, and the deck deliberately keeps them in separate sections.

When & how it's applied. Use the roadmap to organise revision: every slide in the deck belongs to one of the five outline parts, and the two Summary slides (Introduction p.27–28) recapitulate exactly one pillar each. Topic tag: **Software Processes / CI**.

Motivation illustrations: self-driving cars, washing machines, AI

What it is. The Motivation slide grounds its two bullets ("Internet in everything", "Software is everywhere") in three concrete illustrations (Introduction p.8–9):

1. **Self-driving cars** — an annotated diagram of the Google self-driving car (credited on the slide "Source: Google" and "Raoul Rañoa / @latimesgraphics") with five labelled software-driven components: *a laser sensor scans 360 degrees around the vehicle for objects*; *a processor reads the data and regulates vehicle behavior*; *radar measures the speed of vehicles ahead*; *an orientation sensor tracks the car's motion and balance*; and *a wheel-hub sensor detects the number of rotations to help determine the car's location*.
2. **Washing machines** — even ordinary white goods now run embedded software.
3. **AI** — a human head composed of circuitry, annotated with *AI*, *Watson*, and *Google Photos*, standing for software-driven intelligent services.

What it's used for / why it matters. Each example is software embedded in (or behind) a long-lived product, which is exactly the population of systems whose software must be maintained for years or decades after first release. The self-driving car is the richest example: each labelled sensor implies a software subsystem that must *conform* to a hardware interface (conformity), be *changed* as hardware, law and competition change (changeability), and fail *abruptly* rather than gracefully if wrong (discontinuity) — so the one picture quietly previews three of the five essential difficulties presented on the very next slide.

When & how it's applied. Use these as ready-made exam illustrations for "software is everywhere": a car, a washing machine and a photo service are three different industries, all software-dependent, all needing maintenance. Topic tag: **Software Processes / CI**.

The Essential Properties slide — annotated illustrations in detail

What it is. The Essential Properties slide (Introduction p.9) is hand-annotated, and each of the five properties has a specific canonical picture worth knowing in full:

- **Complexity** — a tessellation of many interlocking shapes, annotated "*Polygons*", "*All Shapes*", and "*As needed*" (a small highlighted patch): a software system is a mosaic of many distinct parts with few repeated elements, extended "as needed" rather than uniformly.
- **Invisibility** — a sheet of binary digits (1s and 0s) curving away into a spiral, paired with a separate node-and-edge tree annotated "*Visualization tools*": the raw artifact is unreadable bits, so structure is only recoverable through tools that visualise it.
- **Changeability** — a branch-and-merge arrow diagram whose three lanes are labelled **master**, **develop**, and **topic**: the slide literally illustrates changeability with a Git-style branching picture, a deliberate forward link to the version-control lab (and the same picture family as GitHub-flow feature branches in IntroLab p.1).
- **Conformity** — drawn as a module **M** connected to a module **D** by a constraint labelled *d*. Note the slide's own spelling is "**Comformity**", and the summary slide (Introduction p.27) spells it "**conformatity**" — both are slide typos for *conformity*.
- **Discontinuity** — a padlock with a password field: one wrong character and the lock simply does not open. There is no partial credit, which is precisely non-continuous behaviour.

What it's used for / why it matters. Slide-recall questions often quiz the pictures, not just the words: be able to pair each property with its canonical illustration and explain *why* the picture fits. The master/develop/topic detail is also the first appearance of branching in the course, two slides before the Git lab formalises it.

When & how it's applied. When writing exam answers, spell the property *conformity* (the slide typos should not be reproduced), and use the padlock and branching images as one-line mnemonics. Topic tag: **OO Principles / Software Processes / CI**.

Software Engineering History — Tukey, Kuhn, and Brooks by name

What it is. The Software Engineering History slide (Introduction p.11) is a hand-drawn map anchored by **three named figures**, each starting a thread of the field's history:

1. **John Wilder Tukey (1958)** → "*First Sw Applications*" → staffed by "*Electrical Engineers*" and "*Mathematicians*" → "*Addhoc*" [slide spelling]. Tukey — the statistician credited with the first printed use of the word "software" (the slide dates him 1958) — anchors the birth of software as a distinct artifact, initially built ad hoc by people recruited from neighbouring disciplines.
2. **Thomas Kuhn** → "*Paradigm*" → "*Anomaly*" → "*Complexity*" → "*Addhoc VS Tranined Skills*" [slide spelling]. Kuhn supplies the conceptual vocabulary the whole lecture leans on: a **paradigm** is the shared framework of assumptions a community works within, and an **anomaly** is an observation the prevailing paradigm cannot explain; accumulating anomalies force a paradigm shift. The deck then applies exactly this machinery to software: growing *complexity* exposed the limits of ad hoc craft versus trained skills, and later the requirements-volatility and CHAOS findings are presented as the *anomalies* of the waterfall paradigm.
3. **Fred Brooks (1982)** → "*IBM OS/360*" → "*SW lifespan models*" and "*Waterfall*". Brooks managed IBM's OS/360 operating-system project, the canonical early example of a very large software effort; the slide credits this line of hard-won experience with motivating software life-span models and waterfall-era process thinking.

What it's used for / why it matters. This slide is the deck's only source of *names and dates*, so it is high-value exam material: software separated from hardware in the **1950s** (Tukey, 1958); **paradigm/anomaly** is Kuhn's apparatus, imported from the philosophy of science; and Brooks connects to **IBM OS/360**. It also explains *why* the lecture calls requirements volatility and the CHAOS data "anomalies" — that word choice is a deliberate Kuhn reference, signalling that waterfall's failures are paradigm-breaking observations, not mere inconveniences.

When & how it's applied. Be careful with the Brooks attribution: on this slide Brooks appears for **OS/360**, *not* for "No Silver Bullet" — the essential/accidental terminology of Introduction p.9–10 is Brooksian background that the deck never explicitly attributes (see the Grounding note). In an exam, citing "Kuhn's paradigm → anomaly → shift" as the structure of the lecture's argument is a strong framing move. Topic tag: **Software Processes / CI**.

Course context: SB5-MAI, prerequisites, and outcome

What it is. SB5-MAI is a **5 ECTS** ("0.083 full-time equivalent") master's-level course whose single stated learning outcome is the **maintenance of larger existing software projects** (Introduction p.4). ECTS (European Credit Transfer System) is the EU's standard unit for measuring study workload; 5 ECTS is roughly 125–140 hours of total student effort, so the figure tells you the intended depth and pace.

What it's used for / why it matters. The prerequisites are deliberately heavy and they define what the course is *not* about. They require advanced object-oriented programming (≈12.5 ECTS), software engineering covering process modeling, requirements, analysis, design and architecture (≈20 ECTS), and project team work; the teaching language is English or Danish (Introduction p.4). The purpose of listing these is to draw a line: the course assumes you can already *write* OO software and *design* systems from scratch, so it spends none of its budget teaching those. Its entire contribution is the harder, less-taught skill of **changing software you did not write** — reading an unfamiliar legacy codebase, locating where a change belongs, assessing its ripple effects, and applying it safely.

When & how it's applied. Treat this slide as the scoping contract for the exam: any question is answered in terms of *modifying an existing system* (JHotDraw), never greenfield construction. If an exam answer drifts toward "design a new system," it has missed the course's premise. Topic tag: **Software Processes / CI**.

Assessment: individual portfolio + 7-point scale

What it is. The exam is a **written report on campus**; the **individual portfolio** is the basis for examination and is assessed on the Danish **7-point grading scale** (Introduction p.5). A "portfolio" here means an accumulated set of worked examples produced across the semester rather than a single sit-down test; the 7-point scale is Denmark's national grading scale (12, 10, 7, 4, 02, 00, −3), used so results are comparable across Danish institutions.

What it's used for / why it matters. The portfolio's concrete task is to **refactor JHotDraw features according to Clean Code**, built from **individual mandatory portfolio examples** (Introduction p.6). This is the single most exam-relevant slide because it tells you what the entire course is *for*: applying refactoring and Clean Code principles (`[MC09]`) to a real legacy Java codebase, and being able to *justify* each change in writing. Because the assessment is individual and report-based, the skill being graded is not just "does the code work" but "can you explain the change process you followed and why each step was safe."

When & how it's applied. Every portfolio entry is, in miniature, a pass through the canonical change process the later lectures formalise: a JHotDraw change should travel through Concept Location (find where it goes), Impact Analysis (what else it touches), Prefactoring (clean up first), Actualization (make the change), Postfactoring (clean up after), and Verification (prove it still works) — even though those phase names appear only in later decks. Day-one tooling (Git + Maven + GitHub) exists so that this portfolio work is reproducible, traceable, and reviewable. Topic tags: **Refactoring**, **Clean Code**, **JHotDraw Case Study**.

Motivation: software is everywhere

What it is. The motivation slides make a deliberately simple, almost slogan-like point: "**Internet in everything**" and "**Software is everywhere**" (Introduction p.8–9). It is an observation about the *installed base* — software now runs in phones, cars, medical devices, infrastructure, and consumer goods, not just on computers people think of as computers.

What it's used for / why it matters. This is the economic justification for a dedicated maintenance course. Because software is pervasive *and* long-lived, the vast majority of all software effort — by most industry estimates well over half of total lifetime cost — is spent not on building new ("greenfield") systems but on changing, fixing, adapting, and evolving systems that already exist and run in the field. A course that taught only how to write new code from scratch would therefore be optimising the minority of the work. Maintenance is where the money, the risk, and the hours actually are.

When & how it's applied. Use this as the framing answer to "why study maintenance at all?" It also sets up the staged life-span model later in the deck: pervasive, deployed software inevitably accumulates change

pressure, decays, and must be serviced or reengineered — which is the rest of the lecture. Topic tag: **Software Processes / CI**.

The five essential difficulties (essential properties)

What it is. Software has **essential** difficulties — hardships that are intrinsic to the *nature of software itself* and that no tool, language, or methodology can remove (Introduction p.9). "Essential" is used in the philosophical sense of *of the essence*: properties software has by virtue of being software. The Essential Properties slide names and illustrates five (Introduction p.9):

- **Complexity** — a software system has an enormous number of distinct, interacting parts, and unlike a physical machine it has few repeated elements, so its complexity grows non-linearly with size (illustrated by a dense web of polygons / "all shapes"). *Used for:* explaining why large systems become hard to understand and why bugs hide; it is the root cause of most other difficulties. *Applied:* JHotDraw's many interacting figure, handle, tool, and editor classes are exactly this kind of irreducible complexity you must navigate when locating a change.
- **Invisibility** — software has no natural geometric or visual representation; you cannot "see" its structure the way you see a bridge or a building, because it is pure abstraction with no spatial form (illustrated by an abstract/invisible-structure motif). *Used for:* justifying why we depend on diagrams, models, IDE call-graphs, and visualization tools to reason about code. *Applied:* you read JHotDraw through class diagrams and IDE navigation precisely because the code's "shape" is not directly perceptible.
- **Changeability** — software is under constant pressure to change because it is the *most malleable* part of any system, so whenever a system must adapt, the software is what gets altered (illustrated with branching/merging arrows reminiscent of version-control branches). *Used for:* explaining why a finished program is never really finished, and why maintenance dominates. *Applied:* the whole course — and version control itself — exists to make this constant change disciplined rather than chaotic.
- **Conformity** — software must *conform* to arbitrary external interfaces, standards, hardware, and human institutions that it has no power to redesign; much of its complexity is imposed from outside rather than chosen (illustrated as modules M and D connected by a constraint *d*). *Used for:* explaining why code is full of awkward special cases that exist only to match some external system. *Applied:* JHotDraw conforming to the SVG file format and to Java/Swing GUI conventions is conformity in action.
- **Discontinuity** — small changes to input or to the code can cause disproportionately large, *non-continuous* jumps in behaviour; software does not degrade gracefully the way a physical structure under slight overload does (illustrated by a padlock/security icon, evoking how one wrong bit breaks everything). *Used for:* explaining why "small, safe-looking" edits cause outsized regressions, and why testing/verification is mandatory. *Applied:* it is the reason a one-line refactor in JHotDraw must still be verified, not assumed safe.

What it's used for / why it matters (collectively). These five are the engineer's permanent, non-negotiable problem set. Naming them up front lets the course position every later technique (refactoring, Clean Code, testing, the change process) as a *coping strategy* for an essential difficulty rather than as a way to eliminate it. The exam-critical claim is that these difficulties can be *managed* but never *removed* — there is no silver bullet.

When & how it's applied. These are the classic Brooks "No Silver Bullet" essential difficulties, though the slide does not cite Brooks by name (see Grounding note). The summary slide restates them verbatim: "software engineers are faced with essential difficulties of complexity, invisibility, changeability, conformity, and discontinuity" (Introduction p.27). In an exam, expect to name all five, give a one-sentence explanation of each, and contrast them with accidental properties. Topic tag: **OO Principles / Software Processes / CI**.

Accidental properties

What it is. In direct contrast to the essential difficulties, **accidental properties** are the incidental, *replaceable* concerns of building software: the specific **technologies**, **operating systems**, and "roles of software" that happen to be in fashion in a given era (Introduction p.10). "Accidental" again means the opposite of essential — not "by mistake" but "not of the essence," i.e. true of *this* implementation but not of software in general. The slide illustrates this with a wall of brand logos (GWT, Joomla, Magento, WordPress, AJAX, Silverlight, .NET, PHP, Java, iPhone, Android, SharePoint, Facebook, Sitecore, etc.).

What it's used for / why it matters. The distinction matters because it tells you where engineering effort actually pays off. Accidental properties — framework choices, languages, platforms — turn over every few years (several logos on the slide are already obsolete), so investing your understanding in them yields short-lived returns. The essential difficulties persist across all of them. The teaching point is therefore strategic: maintenance discipline must target the essential problems (managing complexity, enabling safe change), not chase accidental fashion, because a new framework never makes complexity, invisibility, or discontinuity go away.

When & how it's applied. Use this pairing whenever an argument claims a tool "solves" software's hard problems: a faster language or a trendier framework is an *accidental* improvement and cannot touch the *essential* difficulties. In the course itself, Java/Maven/Swing are accidental details of the JHotDraw case study; the transferable skills are the change process and Clean Code, which would apply equally to any stack. Topic tag: **Software Processes / CI**.

Accidental Properties — the logo wall transcribed

What it is. The Accidental Properties slide (Introduction p.10) is a single banner of brand logos with three hand-written annotations classifying them: "**Technologies**" (the banner as a whole), "**Operating systems**" (an arrow pointing at Android), and "**Roles of Software**". The visible logos, row by row: GWT, Drupal, Joomla!, Magento, WordPress, AJAX; Windows Media, Silverlight, Flash (Fl), .NET, PHP, Java; iPhone, Android, SharePoint, Facebook, Sitecore, Zend Framework, ooVoo.

What it's used for / why it matters. The slide's argument is carried by the *datedness* of the wall itself: Silverlight and Flash are discontinued, GWT and Joomla have faded, and the remainder have all changed beyond recognition — while every one of the five essential difficulties on the previous slide applies unchanged to whatever replaced them. The three annotations give the slide's implicit taxonomy of accidental choice: which *technology/framework* you build with, which *operating system/platform* you target, and which *role* the software plays (CMS, e-commerce, social platform, communication tool, …). All three are decisions that get remade every few years; none touches complexity, invisibility, changeability, conformity or discontinuity.

When & how it's applied. A strong exam answer names two or three logos as evidence ("Silverlight and Flash, on the slide as current technologies, are now dead — but the systems built on them still need maintenance, which is an essential concern"). The wall also quietly makes the maintenance case: software routinely *outlives* the accidental technology it was built on, and the migration work that follows is itself maintenance. Topic tag: **Software Processes / CI**.

Software engineering history and the rise of paradigms

What it is. This slide gives the historical arc that produced today's process landscape. Software became a **product in its own right in the 1950s**, when it was "separated from the hardware" and software engineering was "defined as a field on its own" (Introduction p.11, p.14, p.27). Before that, software was an afterthought bundled with the machine; once it became a standalone product with its own economics, it

needed its own engineering discipline. Programmers were initially **recruited from hardware engineers and mathematicians** (the *ad hoc* era) (Introduction p.14).

What it's used for / why it matters. The history is told to set up the central narrative of the lecture: the field responded to the growing difficulty of building software with a *sequence of paradigms* (Introduction p.12) — broad, shared philosophies of how software should be developed. Understanding that paradigms arose as successive *responses to failure* (ad hoc didn't scale → waterfall was tried → waterfall's anomalies → agile) is what makes the later "they co-exist" conclusion meaningful rather than arbitrary.

When & how it's applied. Treat this as the spine onto which the next three concepts hang. In an exam, the "1950s, separated from hardware, recruited from hardware engineers and mathematicians" facts are the canonical framing for the ad hoc era. Topic tag: **Software Processes / CI**.

Paradigm 1 — Ad hoc

What it is. The **ad hoc** paradigm is the earliest mode of software development: *craft without process*. There was no defined method; programmers were drawn "from the ranks of hardware engineers and mathematicians," software had only just separated from hardware in the 1950s, and software engineering was newly defined as its own field (Introduction p.13–14). Each programmer worked in their own style, relying on individual skill rather than any shared, repeatable procedure.

What it's used for / why it matters. Ad hoc is presented as the baseline that everything else improves on. It works for small, short-lived programs written and maintained by one expert, but it does not scale to large or long-lived systems or to teams, because there is no shared structure for understanding, handing off, or evolving the code. Its limitations are precisely the motivation for the next paradigm: when projects grew, the absence of process became the problem.

When & how it's applied. Recognise ad hoc as the answer to "what came before defined processes?" and as the still-present reality of throwaway scripts and tiny one-person tools. The course's emphasis on disciplined version control and a defined change process is, in part, the deliberate opposite of ad hoc. Topic tag: **Software Processes / CI**.

Paradigm 2 — Waterfall and its anomalies

What it is. The **Waterfall** model is a **linear, sequential process** borrowed from **construction and manufacturing**, where work flows downward through fixed phases (requirements → design → implementation → testing → delivery) like water down a series of steps; the slide calls it an "intuitively appealing metaphor" (Introduction p.15). Its defining assumption is that each phase is *completed and frozen* before the next begins — above all, that requirements can be fully fixed up front.

What it's used for / why it matters. Waterfall's purpose was to bring order and predictability to development by front-loading planning, which is genuinely attractive for projects with stable, well-understood requirements (and remains sensible where change is rare and the cost of error is catastrophic). But its core assumption — fixed up-front requirements — is exactly where it breaks for most software. The deck presents two empirical **anomalies** (observed facts the model cannot account for) that puncture it:

- **Requirements volatility (Casper/Capers Jones).** Requirements for IT "change at a rate 2 to 3 % per month" (Introduction p.16). The Agile-Approach slide gives the per-software-type breakdown of monthly requirements change: contract/outsource software **1.0%**, information systems **1.5%**, system software **2.0%**, military software **2.0%**, commercial software **3.5%** (Introduction p.17, slide "Agile Approach"; the slide credits the data to "Strategies for Managing Requirements Creep", C. Jones, 1996, *IEEE Computer*).

Why it matters: over a multi-month waterfall project this compounding drift makes a frozen up-front specification obsolete before delivery — you build, perfectly, the wrong thing. *Applied:* it is the quantitative case for iterating instead of freezing.

- **The Standish Group / CHAOS anomaly.** The **CHAOS Reports**, published yearly since **1994**, track **~50,000 projects worldwide**, scoring success by a then-new definition: **on time, on budget, with a satisfactory result** (Introduction p.17 [slide "Standish Group Anomaly"]). *Why it matters:* the data show large fractions of projects are *challenged* or outright *failed*, directly contradicting waterfall's promise of predictable, on-plan delivery. *Applied:* it is the empirical evidence that the dominant heavyweight process was not actually delivering.

When & how it's applied. In an exam, define waterfall as a linear construction-style process, then *immediately* pair it with its two anomalies (volatility ~2–3%/month; CHAOS failure rates) as the reason iterative methods arose. Do not, however, declare waterfall "dead" — the summary insists the paradigms co-exist. Topic tag: **Software Processes / CI**.

Waterfall's exact phase sequence

What it is. The waterfall diagram (Introduction p.15) shows exactly **five boxes** cascading down-right, each handing off to the next with no arrows back up: **Requirements → Design → Implementation → Testing → Deployment**.

What it's used for / why it matters. Two details of the drawing are exam-relevant. First, the complete enumeration: five phases, ending in *Deployment* — be able to reproduce all five in order. Second, what is *missing*: the waterfall as drawn contains **no maintenance box at all**. The model's world ends at deployment, which is precisely the course's complaint about it — the dominant, decades-long post-delivery life of the software is simply not represented. Contrast this with the V-model (maintenance at the top of the right arm, Introduction p.24) and the prototype model (maintenance as the terminal stage, Introduction p.25), both of which name maintenance explicitly.

When & how it's applied. If an exam question asks "where is maintenance in the waterfall model?", the grounded answer is *nowhere on the slide's diagram* — its absence is the point. Topic tag: **Software Processes / CI**.

The tree-swing cartoon — requirements volatility illustrated

What it is. The Anomaly of Requirements Volatility slide (Introduction p.16) carries the classic five-panel tree-swing cartoon, with these captions in order: "**How the customer explained it**" (a swing with three stacked boards), "**How the project leader understood it**" (a single board lying on the ground under the tree), "**How the analyst designed it**" (a swing cut through the tree trunk), "**How the programmer wrote it**" (a bare rope, unusable), and "**What the customer really wanted**" (a simple tire swing).

What it's used for / why it matters. The cartoon adds a second, distinct argument to the volatility statistic on the same slide. The 2–3 %/month figure says requirements *drift over time*; the cartoon says requirements are additionally *distorted across every role hand-off* — customer → project leader → analyst → programmer — and a linear process like waterfall chains all those hand-offs together with no feedback until the end, so the distortions compound silently. Even with zero drift, a pure hand-off pipeline can deliver a rope when the customer wanted a tire swing.

When & how it's applied. Pair the cartoon with agile's cure: short iterations that put a **Deliverable** in front of the customer and route **Feedback** back into the **Backlog** every cycle (Introduction p.17–18), so a misunderstanding surfaces within one iteration instead of at final delivery. In an exam, the cartoon is the

qualitative half and the Capers Jones / CHAOS numbers are the quantitative half of the same anomaly argument. Topic tag: **Software Processes / CI**.

CHAOS Modern Resolution 2011–2015 — the all-projects table

What it is. The Standish Group Anomaly slide (Introduction p.17, slide "Standish Group Anomaly") reproduces the CHAOS "**MODERN RESOLUTION FOR ALL PROJECTS**" table, year by year:

Outcome	2011	2012	2013	2014	2015
Successful	29%	27%	31%	28%	29%
Challenged	49%	56%	50%	55%	52%
Failed	22%	17%	19%	17%	19%

The slide's footnote defines the metric: the *Modern Resolution* (**OnTime, OnBudget, with a satisfactory result**) of **all software projects from FY2011–2015** within the new CHAOS database. The accompanying bullets: the CHAOS Reports have been published **every year since 1994**, cover **50,000 projects around the world**, and use this (then-new) definition of success factors (Introduction p.17).

What it's used for / why it matters. The numbers, not just the existence of the report, are the anomaly: in *every single year* roughly **70% of all software projects are challenged or failed** (successful never exceeds 31%), and the pattern is stable across five consecutive years — so it is not a bad year but a systemic property of how software was being delivered. That stability is what makes the data paradigm-threatening in Kuhn's sense (Introduction p.11): the prevailing process model predicted controllable, on-plan delivery and reality persistently said otherwise.

When & how it's applied. Keep this table firmly separate from the **agile-versus-waterfall** table (Introduction p.19): this one is *all projects, segmented by year*; the other is *projects segmented by method and size* (and is the one showing agile 39% vs waterfall 11% successful). Mixing the two tables up is an easy exam error. Topic tag: **Software Processes / CI**.

The Capers Jones change-rate table — exact values and source

What it is. The Agile Approach slide (Introduction p.17, slide "Agile Approach") opens with a hand-styled table titled "Software Type / Monthly Rate of Requirements Change":

Software type	Monthly rate of requirements change
Contract or outsource software	1.0 %
Information systems	1.5 %
System software	2.0 %
Military software	2.0 %
Commercial software	3.5 %

The slide credits the data: *From "Strategies for Managing Requirements Creep", C. Jones, 1996, IEEE Computer*. (The earlier volatility slide's "(Casper-Jones)" is a slide-level misspelling of the same author, **Capers Jones**; Introduction p.16.)

What it's used for / why it matters. The per-type values bracket and substantiate the headline claim that requirements change at "2 to 3 % per month": the average sits in that band, with **contract/outsource software the most stable (1.0%)** — plausibly because contracts freeze scope — and **commercial**

software the most volatile (3.5%), because market-facing products chase competitors and customers. The compounding consequence is the real argument: at 2–3% per month, a project that takes 18 months from frozen specification to delivery accumulates on the order of 36–54% requirements churn (simple linear accumulation of the slide's monthly rates), so the system delivered to the original spec is, by then, substantially the wrong system.

When & how it's applied. Quote the exact numbers and the source title in an exam answer — "Jones, *Strategies for Managing Requirements Creep*, IEEE Computer, 1996" — and read the table as the quantitative reason agile re-plans every iteration instead of freezing a long plan. Note the trap: the values are **1.0–3.5 percent per month**, not 10–35 percent. Topic tag: **Software Processes / CI**.

Paradigm 3 — Agile / Iterative approach

What it is. The **Agile** (iterative) paradigm replaces waterfall's single linear pass with a *repeating short cycle*: **Plan → Collaborate → Deliver** (Introduction p.18). Instead of specifying everything once and building once, you build a small increment, get feedback, and re-plan, over and over. The illustrated loop is: a **Backlog** of *Items* feeds an **Iteration** that has a **Daily Review**, producing a **Deliverable** at each **Release**, with **Feedback** flowing back into the backlog (Introduction p.18).

What it's used for / why it matters. Agile exists to *absorb* the very requirements volatility that breaks waterfall. Because each iteration is short and re-planned from the latest backlog, change is expected and welcomed rather than fought; the 2–3%/month drift becomes harmless because you never commit to a long frozen plan. The empirical case is the **CHAOS "resolution by agile versus waterfall"** table (data from FY2011–2015, >10,000 projects) (Introduction p.19):

Project size	Method	Successful	Challenged	Failed
All sizes	Agile	39%	52%	9%
All sizes	Waterfall	11%	60%	29%
Large	Agile	18%	59%	23%
Large	Waterfall	3%	55%	42%
Medium	Agile	27%	62%	11%
Medium	Waterfall	7%	68%	25%
Small	Agile	58%	38%	4%
Small	Waterfall	44%	45%	11%

Agile outperforms waterfall on success rate at every project size, most dramatically on large projects (18% vs 3% successful) — the larger and riskier the project, the bigger agile's relative advantage.

When & how it's applied. Crucially, the summary insists the **three paradigms co-exist** — agile did not erase ad hoc or waterfall; each survives in suitable niches (a tiny script can stay ad hoc; a safety-critical system with stable requirements may still suit a waterfall-like plan) (Introduction p.27). For this course, the agile feedback loop is the conceptual ancestor of the canonical change process: small, verified increments fed by a backlog of change requests (later: GitHub Issues). Each JHotDraw portfolio change is one such iteration. Topic tags: **Software Processes / CI**, with a forward link to **Software Change Process** (the iterative change loop is the spiritual ancestor of the canonical change process).

Software life-span models — overview

What it is. A **life-span model** is a high-level description of the **stages software goes through from conception to death** — analogous to the life cycle of any other product, but with stages that can look very different from one another (Introduction p.20–21). Where a *process paradigm* describes *how you develop*, a *life-span model* describes *what phase the product is in* over its whole existence, from first release through active growth, into decline, and finally retirement.

What it's used for / why it matters. These models exist to give a "simplified and comprehensive view of the entire software engineering discipline" (Introduction p.28) — a single mental picture that locates *where maintenance fits* in a product's life. That placement is the whole point for this course: maintenance is not a separate activity bolted on at the end but a named, dominant stage (or set of stages) in every model. The deck presents four/five models (Introduction p.28): **staged**, **reengineering** (a variant/overlay on the staged transitions), **versioned-staged**, **V-model**, and **prototype**.

When & how it's applied. Use a life-span model to answer "what kind of change is appropriate right now?" — deep evolution, light servicing, or retirement. In the exam, be able to sketch each model and, critically, point to where *maintenance* and *servicing* sit. Topic tag: **Software Processes / CI**.

The product life-cycle curve behind life-span models

What it is. The Life-Span Models overview slide (Introduction p.20–21) pairs its three bullets — stages through which software goes *from conception to death*; stages *may be very different*; stages are *similar to the stages in the life span of other products* — with a classic marketing **product life-cycle curve**: Product Sales on the vertical axis rising and falling over four labelled stages, **Introduction → Growth → Maturity → Decline**.

What it's used for / why it matters. The curve is the "other products" analogy made visible: software life-span models are deliberately borrowed from general product management, where every product is born, grows, peaks, and declines. Mapping the marketing curve onto the staged model is a useful mnemonic — *Introduction* ≈ initial development / first version, *Growth* ≈ Evolution (the product is actively gaining capability and value), *Maturity* ≈ Servicing (the product earns but no longer grows), *Decline* ≈ Phase-out and Close-down. The mapping also carries an economic intuition: just as sales peak in maturity, much of a software product's revenue is earned while it is merely being *serviced*, which is why organisations under-invest in reengineering even as decay accumulates.

When & how it's applied. If asked why the course says "life-span" rather than "lifecycle of a project", this slide is the answer: the unit of analysis is the *product over its whole commercial existence*, not one development effort. Topic tag: **Software Processes / CI**.

Staged model

What it is. The **Staged Model** describes a single product's life as five sequential stages connected by named transitions (Introduction p.21):

1. **Initial development** — builds the *first version* from nothing; there is no prior code to respect.
2. **Evolution** — substantial *evolution changes* (new features, architectural changes) are made *while the team still understands the code* and the design is healthy enough to extend.
3. **Servicing (or Maintenance)** — once *evolution stops*, only smaller *servicing patches* are applied (bug fixes, minor tweaks); the code is no longer deeply restructured, only kept running.
4. **Phase-out** — *servicing is discontinued*; the system still runs in the field but receives no further changes.

5. **Close-down** — *switch-off*; the system is retired and decommissioned.

What it's used for / why it matters. The model's value is the sharp line it draws between **Evolution** and **Servicing**. Evolution is where real, valuable change happens — and where the canonical change process and refactoring belong — because the team can still safely reshape the system. Servicing is a *degraded* mode: the team's understanding (or the code's health) has eroded to the point where only cautious patches are affordable. The transition from one to the other is not a decision so much as a slow loss of capability. This framing tells you *why maintenance work varies in kind*: the same bug fix is cheap in Evolution and risky in Servicing.

When & how it's applied. Locate the course's own work on this model: the portfolio's refactoring of JHotDraw is deliberately **evolution-style** change — restructuring a real legacy system while keeping it healthy enough to keep evolving. In an exam, reproduce the five-stage sequence with its transitions and be ready to argue which stage a given change belongs to. Topic tags: **Software Processes / CI**, **Software Change Process**, **Technical Debt** (the slide into servicing is driven by accumulated decay).

Reengineering (avoiding code decay)

What it is. The **Reengineering** slide overlays the staged model with the *forces* that drive software between its stages (Introduction p.22). The natural, gravity-like drift is **Evolution → Servicing**, caused by **code decay**: as the codebase accumulates patches, duplication, and shortcuts, it gradually becomes too tangled and risky to evolve, so the team retreats to cautious servicing. **Reengineering** is the deliberate counter-force — the arrow that goes *back* from **Servicing → Evolution** — restoring the code's internal health (for example through refactoring) so that real evolution can resume.

What it's used for / why it matters. This single slide is the conceptual justification for the entire course. It says, in process terms, that without active intervention software *inevitably decays* out of the productive Evolution stage and into the impoverished Servicing stage — and that **reengineering/refactoring is the only thing that reverses that decay**. The transitions shown make the dynamics explicit: *first running version* (Initial → Evolution), *software changes* (Evolution self-loop), *code decay* (Evolution → Servicing), *reengineering* (Servicing → Evolution), *servicing patches* (Servicing self-loop), *servicing discontinued* (Servicing → Phase-out), and *switch-off* (Phase-out → Close-down) (Introduction p.22). "Code decay" is the same phenomenon later framed as **technical debt**: the gradual interest you pay for past shortcuts.

When & how it's applied. Whenever you refactor JHotDraw in the portfolio, you are performing reengineering in the sense of this slide — spending effort to push the code from a decaying state back toward a healthy, evolvable one, *before or alongside* adding the feature. In an exam, pair "code decay" and "reengineering" as opposing forces and name refactoring as the mechanism of reengineering. Topic tags: **Technical Debt**, **Refactoring**, **Software Change Process**.

Versioned staged model

What it is. The **Versioned Staged Model** generalises the staged model to the realistic case where **multiple shipped versions run in parallel**, each living out its own staged life (Introduction p.23). A central trunk of ongoing **evolution** repeatedly spawns versions: *Initial development → Evolution Version 1 → Servicing Version 1 → Phase-out Version 1 → Close-down Version 1*; meanwhile *evolution of a new version* branches off to *Evolution Version 2 → Servicing Version 2 → Phase-out Version 2 → Close-down Version 2*, and so on (*Evolution Version …*) (Introduction p.23). The key structural feature is that a *new* evolution branch is started *before* the old one is closed down, so several versions are alive at once.

What it's used for / why it matters. This model captures how real commercial software is actually maintained. Vendors don't ship one version and then replace it cleanly; they keep multiple releases supported simultaneously — one in active evolution, an older one in servicing, an even older one being phased out. It explains why maintenance teams must back-port fixes across several living versions and why version control's branching is essential infrastructure: each parallel version is, quite literally, a long-lived branch with its own life cycle.

When & how it's applied. Think of any product with "current," "LTS," and "legacy" releases supported at the same time — that is the versioned-staged model. It connects directly to the Git branching introduced in the lab: parallel supported versions are parallel branches, each receiving the kind of change appropriate to its stage. Topic tag: **Software Processes / CI**.

V-model

What it is. The **V-Model** is a life-span/process model that reshapes the linear sequence into a "V": each *constructive* activity on the descending left-hand side is explicitly **paired with** a corresponding *verification* activity on the ascending right-hand side (Introduction p.24):

- **requirements ↔ functional testing**
- **system design ↔ system testing**
- **unit design ↔ unit testing**
- bottom of the V: **implementation** (the single point where the two sides meet)
- above functional testing: **maintenance** (the post-delivery stage at the top of the right arm)

The dashed horizontal links show that each level of testing exists *specifically to validate* the design level opposite it.

What it's used for / why it matters. The V-model's purpose is to make **testing a first-class, planned activity that is designed at the same time as the thing it tests** — not an afterthought tacked on at the end. By tying functional testing to requirements, system testing to system design, and unit testing to unit design, it guarantees that every level of construction has a defined way to be verified, and that test plans can be written early (while you're designing) rather than improvised late. Placing **maintenance** at the very top frames it, again, as the dominant post-delivery phase.

When & how it's applied. The V-model foreshadows the course's heavy emphasis on testing and on the **Verification** step of the canonical change process: every portfolio change must be paired with a way to prove it is correct, exactly as each design level here is paired with a test level. When you write a unit test for a refactored JHotDraw class, you are instantiating the unit-design ↔ unit-testing pairing of this model. Topic tags: **Software Testing**, **BDD / Verification**, **Software Processes / CI**.

Prototype model

What it is. The **Prototype Model** inserts an early, often throwaway, *learning* artifact into the life span specifically to nail down requirements *before* committing to real design and implementation (Introduction p.25): **requirements → prototype → corrected requirements → design → implementation → maintenance**. The prototype is a quick, partial build whose purpose is to be shown to stakeholders so misunderstandings surface early.

What it's used for / why it matters. It is a direct countermeasure to the requirements-volatility problem at the *front* of a project: rather than freezing requirements you only half-understand, you build a cheap prototype, learn from the reactions to it, and feed the resulting **corrected requirements** into the real

design. This reduces the risk of building the wrong system — the same risk that breaks waterfall — by buying understanding early and cheaply. Like every model in this deck, it ends in **maintenance**, reinforcing that maintenance is the universal terminal (and longest) stage.

When & how it's applied. Use the prototype model whenever requirements are unclear or contested at the outset and the cost of a wrong guess is high. The throwaway prototype is the disposable cost of learning; the corrected requirements are the durable output. Topic tag: **Software Processes / CI**.

What the Summary slides single out

What it is. The deck closes with two summary slides, one per conceptual pillar. **"Software Engineering"** (Introduction p.27) states four takeaways: software has been *a product since the 1950s*; software engineers face the *essential difficulties* of complexity, invisibility, changeability, conformity ("conformatity" in the slide's spelling) and discontinuity; there are *three different paradigms* — **Ad hoc**, **Waterfall**, **Iterative** — and *"The different paradigms co-exists"* [slide wording]. **"Software Life-Span Models"** (Introduction p.28) states that life-span models offer *a simplified and comprehensive view of the entire software engineering discipline* and lists exactly four models: **Staged model**, **Versioned staged model**, **V-model**, **Prototype**.

What it's used for / why it matters. A deck's own summary is its author's priority list, i.e. the highest-probability exam content. Two details deserve attention. First, the paradigm summary names the third paradigm "**Iterative**" (where the body slides say "Agile") — treat the terms as synonyms in this course. Second, the life-span summary lists **four** models, *not five*: **reengineering is not listed as a separate model**, because it is an overlay on the staged model's transitions (the Servicing → Evolution arrow, Introduction p.22), not a model of its own. An exam answer that lists "the life-span models" should give the summary's four and may then *add* that the reengineering slide extends the staged model.

When & how it's applied. Memorise both summary slides verbatim — they are short, and every clause in them is a one-mark recall item. Topic tag: **Software Processes / CI**.

Why version control? (six reasons)

What it is. A **Version Control System (VCS)** is a tool that records the complete history of a project's files over time and coordinates changes among multiple people. The Git Lab opens by justifying VCS with six concrete benefits (Git Lab p.3):

1. **Enforce discipline** — a defined commit/update workflow imposes a repeatable process on a team, so changes happen in an orderly, agreed way rather than ad hoc.
2. **Archive versions** — every past state of the project is preserved and can be retrieved exactly, so you can always return to any prior point.
3. **Maintain historical information** — the system records *who* changed *what*, *when*, and (via commit messages) *why*, giving an audit trail and an explanation of the project's evolution.
4. **Enable collaboration** — multiple developers can work on the same codebase at once without silently overwriting each other's work.
5. **Recover from accidental deletions or edits** — mistakes are reversible; a deleted file or a bad edit can be restored from history.
6. **Conserve disk space** — successive versions are stored efficiently as *deltas* (the differences between versions) rather than as full copies of everything.

What it's used for / why it matters. For a maintenance course this list is foundational rather than incidental. Maintenance *is* a long sequence of changes to existing software, and a VCS is the substrate that makes that sequence **traceable, recoverable, and collaborative**. Reasons 2 and 3 (archive + history) are

what let you understand *how* a legacy system reached its current state; reason 4 (collaboration) is what lets a team work the same codebase; reason 5 (recovery) is the safety net that makes bold refactoring affordable; reason 1 (discipline) is what turns chaotic editing into a process you can grade.

When & how it's applied. Every change to JHotDraw in this course is made *through* version control, so history and traceability are guaranteed for the portfolio and for the exam. When you later "file a change request as an Issue" and "commit your initial code," you are exercising exactly these six benefits. Topic tag: **Version Control / Git**.

The Version Control Lab deck: structure, opening picture, and the two demos

What it is. The Git Lab deck is organised as a funnel from tool-agnostic concepts to the concrete course project. Title: "Version Control — Lab" (Git Lab p.1). The opening **Version Control** picture (Git Lab p.2) shows four developers, each exchanging documents bidirectionally with a central database cylinder labelled "**(VCS)**" — the version control system sits between *every* developer and the shared project state, mediating all exchange of changes. Then come the generic VCS slides (why, actions, workflow, don'ts, types; Git Lab p.3–7), a "**GIT — Lab**" section divider (Git Lab p.8), the Git-specific slides (about, install, workflow; Git Lab p.9–11), two live demonstrations — "**GIT Demo 1: Basics**" (Git Lab p.12) introducing the *local*-repository workflow slide, and "**GIT Demo 2: Advanced**" (Git Lab p.14) introducing the *remote*-repository workflow slide — a "**GIT Hub**" section divider (Git Lab p.16), a "**LAB**" divider (Git Lab p.17), the Projects slide naming JHotDraw (Git Lab p.18), and finally the LAB TODOs (Git Lab p.19).

What it's used for / why it matters. The Basics/Advanced split of the demos is itself didactic and mirrors the four-area model: everything **local** — `status`, `diff`, `commit`, `show` — is "Basics" (Demo 1, Git Lab p.12–13), and everything involving a **remote** — `clone`, `pull`, `push` — is "Advanced" (Demo 2, Git Lab p.14–15). That split is a direct restatement of decentralization: a Git user can be fully productive in Demo-1 territory offline, and only Demo-2 commands touch the network. The p.2 opening picture is the same diagram as the centralized half of the Types-of-VCS slide (Git Lab p.7); the deck deliberately starts from the simpler central-hub mental model before complicating it with decentralization.

When & how it's applied. When revising, rehearse the two demos as two checklists: Demo 1 = edit → status → diff → commit → show, entirely offline; Demo 2 = clone once, then pull/push to synchronise. Topic tag: **Version Control / Git**.

Essential VCS actions and the central-repository workflow

What it is. Stripped to essentials, every VCS provides three core **actions** (Git Lab p.4): **Add** (stage a new or changed file so the system will track it for the next recorded version), **Commit** (record the staged change into the repository together with a descriptive message), and **Update** (pull the latest recorded state from the repository into your working copy). The generic **VCS Workflow** diagram makes this concrete with two developers each editing the same file `foo.txt`: each one **add+commit**s their changes up to a shared **central repository**, and **update**s to receive the other developer's changes, so the repository is the single point at which everyone synchronises (Git Lab p.5).

What it's used for / why it matters. These three actions are the irreducible vocabulary of *any* version control tool — Git's richer command set is just an elaboration of Add/Commit/Update. Understanding them at this abstract level first means you can recognise the same operations in any VCS you meet, and it isolates the one idea that everything else builds on: changes flow *into* a shared store (commit) and *out of* it (update), and the store is the team's single source of truth. The central-repository picture also sets up the contrast that comes next — what changes when *every* developer has a full repository rather than just a working copy.

When & how it's applied. When you later run Git's `add`, `commit`, and `pull`, you are performing exactly these three essential actions; the central repository in the diagram becomes GitHub ("origin") in the JHotDraw setup. The two-developers-on-`foo.txt` scenario is the minimal model of the team collaboration you'll do on feature branches. Topic tag: **Version Control / Git**.

"Don't do it" — what NOT to commit

What it is. A cautionary slide warns against putting the wrong kinds of artifact under version control (Git Lab p.6): **do not** commit **executables** (e.g. an `.EXE`) or **large binary files** (illustrated as a 2 GB file). The repository should hold **VCS source files** — the human-authored source from which everything else is generated — *not* generated or **local version files** (build output).

What it's used for / why it matters. The guiding principle is **commit source, not build output.** This matters for two reasons grounded in how a VCS works. First, version control stores changes efficiently as deltas of *text* (reason 6 above); large binaries and compiled executables don't diff or compress well, so they bloat the history permanently and slow every clone forever. Second, generated artifacts are *reproducible* from source by the build tool, so committing them is redundant *and* dangerous: they go stale, they conflict badly during merges, and they can mask which source actually produced them. Keeping only source ensures the repository stays small, mergeable, and authoritative.

When & how it's applied. In the JHotDraw/Maven setup this is precisely why the Maven `target/` directory (compiled `.class` files, packaged JARs) is excluded via `.gitignore` and never committed — it is regenerated by `mvn clean install` whenever needed. The 2 GB-file illustration is the extreme case (think bundled datasets or media) that should live outside version control entirely. Topic tag: **Version Control / Git**.

Centralized vs decentralized VCS

What it is. Version control systems come in two architectural **types** that differ in *where the full history lives* (Git Lab p.7):

- **Centralized** — there is a single central repository that holds the authoritative history, and every developer talks to it directly. Each developer's working copy is just the current files; it contains **no full history** of its own, so most operations require contact with the central server. *Examples:* CVS, Subversion (SVN).
- **Decentralized / distributed** — every developer's clone is a **complete repository** with the **entire history** stored locally. Repositories synchronise with each other peer-to-peer or, in practice, through a shared remote. *Example:* Git.

What it's used for / why it matters. The architecture determines what you can do offline and how resilient the project is. In a centralized system, committing, viewing history, and branching all need the server; if the server is down or unreachable, you are largely stuck, and if it is lost, history can be lost with it. In a decentralized system, you can commit, inspect full history, branch, and diff entirely **locally and offline**, because you already hold the whole repository; the shared remote becomes a synchronisation point rather than a single point of failure. This decentralization is exactly what produces Git's defining behaviour — that `commit` is local and a separate `push` is needed to share it.

When & how it's applied. Git is explicitly the **decentralized** choice, created by **Linus Torvalds** (slide spelling "Linus Thorvalds") to manage **large projects efficiently** — its original purpose was the **Linux** kernel, whose huge, globally distributed contributor base needed exactly this offline-capable, fast-branching model (Git Lab p.9). Installation is via package managers (Linux package; macOS via Xcode/package;

Windows installer) from `http://git-scm.com/downloads` (Git Lab p.10). In the course you experience decentralization directly: each team member's clone of the JHotDraw fork is a full repository they commit to locally before pushing to the shared GitHub origin. Topic tag: **Version Control / Git**.

Installing Git — per-platform notes

What it is. The Install GIT slide (Git Lab p.10) gives one installation route per platform: **Linux** — install via the distribution's package manager ("Package"); **Mac OS** — via **XCode** (whose command-line tools bundle Git) or a package; **Windows** — a package with a graphical installer. The canonical download source for all platforms is `http://git-scm.com/downloads`.

What it's used for / why it matters. The slide's point is that Git is a small, freely available, cross-platform tool with no server component required — a direct consequence of decentralization (Git Lab p.7, p.9): installing the single client program gives you the *complete* system, including full local history, branching and diffing, with no infrastructure to stand up. Contrast a centralized VCS, where a usable installation also implies a central server somebody must run. For the course this means every student machine is a complete version-control environment from minute one; the only shared infrastructure (GitHub) is rented, not installed.

When & how it's applied. Practically: install Git before the first lab, verify it on the command line, then proceed to `git clone` of the team's JHotDraw fork (Git Lab p.15; IntroLab p.1). The official `git-scm.com` source matters because labs assume a current Git with the standard command set used on the workflow slide (Git Lab p.11). Topic tag: **Version Control / Git**.

The Git four-area model and command map

What it is. The central Git Lab diagram models Git as **four areas**, with commands as the operations that move content between them (Git Lab p.11). The four areas are the staging "pipeline" a change passes through on its way from your disk to the shared server:

1. **Workspace (working dir)** — your actual files on disk, the ones you edit. This is the only area you change directly with an editor.
2. **Index (stage)** — a staging area holding exactly the set of changes you have selected for the *next* commit. It lets you compose a commit deliberately rather than committing everything at once.
3. **Local repository (HEAD)** — your committed history, stored *locally* (this is the decentralized part). `HEAD` is a pointer to your latest local commit.
4. **Remote repository** — the shared server copy (e.g. on GitHub, conventionally named **origin**) that teammates pull from and push to.

Command map (Git Lab p.11):

Command	Moves / does
`clone`	remote repository → workspace (full local copy of everything)
`add (-u)`	workspace → index/stage (`-u` stages already-tracked modified files)
`commit`	index/stage → local repository (HEAD)
`push`	local repository → remote repository
`fetch`	remote repository → local repository (download, no merge)
`merge`	local repository → workspace (integrate fetched/other commits)
`pull`	remote repository → workspace (`fetch` + `merge` in one step)

Command	Moves / does
`diff HEAD`	compare workspace against the last commit (HEAD)
`diff`	compare workspace against the index/stage

What it's used for / why it matters. This four-area model is *the* mental model that makes Git's behaviour predictable instead of magical. The whole reason Git has a separate **index** is to give you control over *what exactly* goes into each commit, so you can record clean, logically-grouped changes rather than a dump of everything you touched — which is what makes a maintenance history readable later. The separation of **local repository** from **remote** is the decentralization in action: it is why `commit` (records locally) and `push` (publishes to the team) are *two different operations*, and why teammates see nothing until you push. Each command in the table is simply a named arrow between two of the four areas — once you know which areas a command connects, you know what it does.

When & how it's applied. Understanding this model — especially that `commit` is *local* and `push` is what shares it, and that `pull` = `fetch` + `merge` — is the single most exam-relevant Git concept. In the JHotDraw workflow you traverse all four areas every change: edit (workspace) → `add -u` (index) → `commit` (local HEAD) → `push` (origin on GitHub), with `diff` / `diff HEAD` letting you inspect the gaps between areas before you commit. Topic tag: **Version Control / Git**.

Git command semantics in depth — the easily confused pairs

What it is. A close reading of the Git Workflow diagram (Git Lab p.11) resolves the command pairs students most often mix up, because each command is drawn as an arrow (or comparison bar) between specific areas:

- `add` **vs** `add -u` — the slide writes `add(-u)`. Plain `add <files>` stages the named files (new or modified) from workspace into the index; the `-u` form stages the modifications and deletions of **already-tracked** files only, never untracked new files. Both end in the same place: the index, area 2.
- `commit` **vs** `push` — `commit`'s arrow runs index → local repository (HEAD); `push`'s arrow runs local repository → remote repository. They are different arrows touching different areas: `commit` records, `push` publishes. Nothing reaches teammates until the second arrow is taken.
- `fetch` **vs** `pull` — `fetch`'s arrow ends at the **local repository**: remote commits are downloaded and stored, but the workspace files are untouched, so nothing you can open in an editor has changed yet. `pull`'s bar spans **all the way to the workspace** because `pull` = `fetch` + `merge` in one step. If you only fetched, you must still `merge` (arrow: local repository → workspace) to see the changes.
- `diff` **vs** `diff HEAD` — both are comparison bars anchored at the workspace, but they reach different distances: `diff` compares workspace against the **index** (showing only *unstaged* edits — what you changed but have not yet `add`ed), while `diff HEAD` compares workspace against the **last commit** (showing *all* changes since HEAD, staged and unstaged together). After `git add -u`, plain `diff` shows nothing while `diff HEAD` still shows your edits — a classic confusion.
- `clone` **vs** `pull` — `clone`'s arrow spans the *entire* diagram, remote repository → workspace, because cloning populates every local area at once: full history into the local repository and a checked-out working copy into the workspace, with the remote linked as `origin` (Git Lab p.11, p.15). `pull` only transports *new* commits into an existing clone. You clone once per machine; you pull continually.
- `status` **vs** `diff` **vs** `show` (Git Lab p.13) — three levels of inspection: `git status` answers *which files* changed; `git diff [files]` answers *which lines* changed; `git show` displays the *last commit* you recorded. The Local Repositories slide also shows the one-time creation prelude in its left margin (`mk…` / `cd …` / `gi…` — the `mkdir` / `cd` / `git init` sequence, partially cut off by the overlay box).

What it's used for / why it matters. Every one of these distinctions is a direct consequence of the four-area architecture, so none of them needs memorising in isolation: identify which two areas a command connects and its behaviour follows. The pairs above are exactly where one-word exam questions hide ("does `fetch` change your files?" — no; "what does `diff` show after staging?" — nothing; "is `pull` atomic-magic?" — it is `fetch` + `merge`).

When & how it's applied. In the JHotDraw labs, the practical rhythm is: `clone` once; per change `status` → `diff` → `add -u` → `commit` → `diff HEAD` to confirm a clean state → `push`; and `pull` before starting work so the merge happens early and small. Topic tag: **Version Control / Git**.

Local repository workflow (a possible everyday loop)

What it is. The Local Repositories slide sketches "a possible workflow (this is just an example!)" — the routine inner loop of working in your *local* repo before any sharing happens (Git Lab p.13): do **some editing**, then run `git status` to **see which files you changed**, `git diff [files]` to **see exactly what changed inside them**, and finally `git commit -a [-m <message>]` to **record the changes** into local history. For inspection it also shows `git status`, `git diff`, and `git show (last commit)` to view the most recent commit (Git Lab p.13). The slide also implies the one-time create-a-repo prelude (`mkdir` / `cd` / `git init`-style steps, partly cut off at the slide margin).

What it's used for / why it matters. This loop is the disciplined "look before you record" habit that turns committing from a careless act into a deliberate one. `git status` tells you the *scope* of your change (which files), `git diff` tells you the *content* (which lines), and only then do you commit — so you always know precisely what you are putting into history, and you catch stray edits or debug code before they pollute the record. Because the commit is local (area 3 of the four-area model), this whole loop runs offline and costs nothing, so it should be done frequently and in small increments. `git show` lets you immediately review what you just recorded.

When & how it's applied. This is the everyday rhythm for each JHotDraw portfolio change: edit a figure class, `git status` to confirm only the intended files changed, `git diff` to read the change, commit with a message explaining *why*, and `git show` to verify. Skipping the inspection step is a named pitfall below. Topic tag: **Version Control / Git**.

Remote repository workflow

What it is. The Remote Repositories slide covers the *outer* loop — working with a shared server copy that the team synchronises through (Git Lab p.15):

- **Copy a repository:** `git clone <repository>`, where `<repository>` is a URL (`file`, `http`, `ssh`, …). Clone **creates a complete local copy** of the entire repository (all files and full history) and **links it to the remote**, which it names **origin** by default; you can also link to a `<repository>` later if you started locally.
- **Receive changes:** `git pull` — bring teammates' published changes down into your workspace.
- **Send changes:** `git push` — publish your local commits up to the remote so teammates can get them.

What it's used for / why it matters. This is the layer at which *collaboration* (VCS benefit #4) actually happens. The local loop above is private to you; nothing is shared until you reach this outer loop. `clone` is how you *join* a project (and, because it copies the full history, how decentralization is established on your machine); `push` and `pull` are the only two operations that cross the boundary between your local repository and the shared one. The named remote **origin** is simply a convenient alias so you don't retype the URL every

time. Keeping the inner (local: status/diff/commit) and outer (remote: clone/pull/push) loops distinct is what prevents the classic confusion of thinking a `commit` has been shared.

When & how it's applied. In the course you `git clone` your team's JHotDraw fork to start, then for each feature you `git push` your branch to origin on GitHub and teammates `git pull` to integrate it. These three commands (clone/pull/push) are the entire interface between your work and the rest of the team. Topic tag: **Version Control / Git**.

GitHub and the course case-study project

What it is. GitHub is a web platform that *hosts* remote Git repositories and adds collaboration features on top of them — forks, pull requests, and **Issues** (a tracker for bugs and change requests) (Git Lab p.16, title slide). It is introduced as the concrete "remote repository / origin" that the workflow above pushes to and pulls from. The course **project** hosted there is **JHotDraw** (Git Lab p.18) — a Java drawing-editor framework that serves as the recurring case study for the entire course.

What it's used for / why it matters. GitHub turns the abstract "shared remote" into a real, addressable place with team features that the course relies on. Two GitHub capabilities are load-bearing here: **forks** (your team's own server-side copy of JHotDraw to push to, without needing write access to the original) and **Issues** (where a change request is recorded *before* code is written — the very first, lightweight instance of the course's **Initiation** phase). JHotDraw itself is chosen because it is a real, non-trivial legacy Java codebase *and* a well-known teaching vehicle for design patterns, so it doubles as material for later lectures.

When & how it's applied. Each team **forks** JHotDraw on GitHub, clones the fork, works on feature branches, pushes back to the fork, and **files change requests as Issues** (Git Lab p.19). JHotDraw is documented in the course literature (`[Kai01]` "Become a programming Picasso with JHotDraw", `[Kir01]` "JHotDraw Pattern Language", `[Ran11a]` / `[Ran11b]` JHotDraw 7 docs), which you will draw on when those design-pattern lectures arrive. Topic tags: **Version Control / Git**, **JHotDraw Case Study**.

LAB TODOs — the four tasks verbatim

What it is. The closing LAB TODOs slide (Git Lab p.19) assigns exactly four tasks, quoted here in slide order:

1. "**Clone remote repository from GitHub**" — perform the `git clone` of the team's JHotDraw fork.
2. "**Get familiar with Git commands** — see this slide for an overview" — the hyperlinked "this slide" points back to the Git Workflow four-area diagram (Git Lab p.11), making that diagram the official command reference for the lab.
3. "**Commit your initial project code using the Git commands add, commit, pull, push…**" — run the full pipeline at least once so every team member has exercised all four areas.
4. "**Input change request as an Issue at GitHub**" — record a change request in the fork's GitHub Issue tracker.

What it's used for / why it matters. The four TODOs are a deliberate traversal of everything the deck taught: TODO 1 exercises the remote workflow (`clone` , Git Lab p.15), TODO 2 anchors the four-area mental model, TODO 3 exercises the local *and* remote pipelines end-to-end (`add` → `commit` → `pull` → `push`), and TODO 4 steps beyond Git into GitHub's collaboration layer. TODO 4 is also conceptually the most important for the course: a change request filed as an Issue *before* code is touched is the first concrete instance of the change-process **Initiation** phase that later lectures formalise — the lab makes you practise starting a change with a recorded request, not with an editor.

When & how it's applied. Treat the four TODOs as the lab's acceptance test: a team is "done" when its fork contains each member's initial commit pushed to GitHub and at least one filed Issue. Topic tags: **Version Control / Git**, **Software Change Process**.

JHotDraw Connection

JHotDraw is introduced here as the **course-long case study** and the target of all portfolio work. Three concrete touch-points in this lecture:

1. **Portfolio task.** The exam portfolio is to **refactor JHotDraw features according to Clean Code** with individual mandatory examples (Introduction p.6). Every later change-process lecture is applied to JHotDraw, so day-one setup must succeed.
2. **The project slide.** The Git Lab's "Projects" slide names **JHotDraw** as *the* project (Git Lab p.18), and "LAB TODOs" tells students to clone it, learn Git, commit their initial code, and **file a change request as an Issue on GitHub** (Git Lab p.19) — the first taste of course **Initiation**: a change request entering the system.
3. **Build/run bootstrap (IntroLab).** The IntroLab "checks out the CASE study source code". Steps (IntroLab p.1): each **team forks** the project repository `[JHotDraw]` on GitHub; members then follow **GitHub flow**, each working on **their own feature branch**; install the **Maven build system 3.8.x on JDK 11**; from the project root run `mvn clean install -DskipTests`; then from `jhotdraw-samples-misc` run `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` — after which the **JHotDraw GUI** (the SVG sample editor) appears.

Together these establish JHotDraw + Maven + Git/GitHub as the fixed technical environment for the whole course. Topic tags: **JHotDraw Case Study**, **Version Control / Git**, **Software Change Process**.

JHotDraw in the course literature

The literature list devotes more entries to JHotDraw than to any other single topic — six references ([Litt] Literature List p.1):

- **[Kai01]** Wolfram Kaiser, *Become a programming Picasso with JHotDraw* (2001, HTML) — the classic hands-on tutorial for learning the framework.
- **[Kir01]** Douglas Kirk, *JHotDraw Pattern Language* (2001, HTML) — documents the design patterns the framework is built from; the bridge to the design-pattern lectures.
- **[Ran11a]** Werner Randelshofer, *JHotDraw 7 Documentation* (2011, HTML) and **[Ran11b]** *The JHotDraw 7 Handbook* (2011, PDF) — the authoritative documentation for the JHotDraw 7 generation the course uses.
- **[Pav11]** Nikolaidis Pavlos, *Software Requirements Specification for JHotDraw* (2011, PDF) — an SRS for the case study, useful when phrasing change requests (GitHub Issues) against documented requirements.
- **[Sav01]** Jolita Savolskyte, *Review of the JHotDraw framework* (2001, PDF) — an external review/critique of the framework.

The density of this cluster confirms what the Projects slide implies (Git Lab p.18): JHotDraw is not a passing example but the course's permanent laboratory — there is dedicated reading for learning it ([Kai01]), understanding its design ([Kir01], [Ran11a/b]), specifying changes to it ([Pav11]), and judging it critically ([Sav01]). Topic tags: **JHotDraw Case Study**, **Software Change Process**.

Worked Example / Process Walkthrough

Goal: get the JHotDraw case study running and under your own version control, the way the IntroLab + Git Lab prescribe. This is the concrete day-one workflow combining IntroLab (p.1), the Git workflow (Git Lab p.11), and the remote workflow (Git Lab p.15). Each step below maps to a Key Concept above, so you can see the theory being applied.

1. **Fork & clone (Initiation / setup).** On GitHub, the team creates a **fork** of the `[JHotDraw]` project repository — a server-side copy the team can push to (IntroLab p.1). Each member then makes a complete local copy: `git clone <fork-url>` — creates a full local repository (all history) linked to the remote **origin**, putting decentralized version control in place on each machine (Git Lab p.15).
2. **Build with Maven.** Install **Maven 3.8.x on JDK 11**, go to the project root and run: `mvn clean install -DskipTests` (IntroLab p.1) — `clean` deletes old build output, `install` compiles and installs all modules into the local Maven cache, and `-DskipTests` skips the test suite for this first *smoke build* (you just want to confirm it compiles, not run everything yet).
3. **Run the GUI.** From the `jhotdraw-samples-misc` folder run: `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` (IntroLab p.1) — launches the named main class so the **JHotDraw SVG sample GUI** opens, confirming a working end-to-end environment.
4. **Make and inspect a change (local loop).** Edit a file, then run the disciplined inspect-then-record loop: `git status` (which files changed) → `git diff [files]` (what changed, line by line) → `git add -u` (stage tracked changes into the index) → `git commit -a -m "<message>"` (record to local HEAD with a reason) (Git Lab p.11, p.13). Use `git show` to review the commit you just made (Git Lab p.13). This traverses areas 1→2→3 of the four-area model entirely locally.
5. **Branch per feature (GitHub flow).** Each team member works on **their own feature branch** so two people's work never collides on the same line (IntroLab p.1) — branching is the mechanism by which a VCS delivers the *collaboration* and *discipline* it exists to provide (Git Lab p.3).
6. **Share work.** `git push` your branch to origin (crossing from local repo to remote, area 3→4); teammates `git pull` (= `fetch` + `merge`) to integrate it into their own workspaces (Git Lab p.11, p.15).
7. **Record the change request.** File the change as an **Issue on GitHub** (Git Lab p.19) — the course's first instance of the canonical **Initiation** phase: a change request is captured in the backlog *before* any code is touched, exactly like the agile backlog item that feeds an iteration.

This walkthrough is the practical embodiment of the lecture: an iterative (agile) change loop, executed through version control, on the JHotDraw case study.

The two-developer foo.txt scenario, re-expressed in Git

The generic VCS Workflow slide (Git Lab p.5) shows two developers collaborating on one file, `foo.txt`, through a central repository. Translating that picture into the Git commands of the four-area model (Git Lab p.11) makes both diagrams concrete and exposes exactly where Git refines the generic model:

1. **Developer A creates/edits** `foo.txt` and performs the slide's "*add* + *commit*" up to the central repository. In Git this is *three* steps because of decentralization: `git add foo.txt` (workspace → index), `git commit -m "explain why"` (index → local HEAD), and `git push` (local HEAD → remote/origin). The generic picture's single "commit to centre" arrow is Git's `commit` **plus** `push`.
2. **Developer B performs "update"** to receive `foo.txt`. In Git: `git pull`, which is `fetch` (remote → B's local repository) followed by `merge` (local repository → B's workspace). The generic "update" arrow is Git's `pull` = `fetch` + `merge`.

3. **Developer B edits the file** — the slide marks the changed version with a prime, `foo.txt'` — and commits it back ("*commit*" arrow): `git add -u`, `git commit`, `git push`.
4. **Developer A updates** ("*update*" arrow, receiving `foo.txt'`): `git pull`, after which A's workspace holds B's revision.

The central database cylinder of the generic slide is, in the course setting, the team's JHotDraw **fork on GitHub**, addressed as **origin** after `git clone` (Git Lab p.15; IntroLab p.1). The prime notation (`foo.txt` vs `foo.txt'`) is the slide's way of marking *successive versions of the same file* — exactly what the VCS archives as history (benefit #2, Git Lab p.3) and stores efficiently as deltas (benefit #6). If both developers had edited `foo.txt` concurrently on the same lines, the integration point would be B's or A's `merge` step — which is why the course has each member work on **their own feature branch** (GitHub flow, IntroLab p.1), keeping concurrent edits apart until integration is deliberate. Topic tag: **Version Control / Git**.

Definitions & Terminology

Each row gives a compact, exam-ready gloss: *what the term is*, and where useful *what it is for*.

Term	Definition	Source
Software maintenance	Changing, fixing, adapting and evolving larger existing software projects *after* initial development; the dominant, lifelong activity in a system's life span and the entire subject of this course.	Introduction p.4
Essential difficulties	The inherent, irreducible hardships intrinsic to software itself — complexity, invisibility, changeability, conformity, discontinuity — that can be *managed* but never *removed* by any tool or method.	Introduction p.9, p.27
Complexity (essential)	The inherently large number of distinct, interacting parts in software; the root difficulty that makes systems hard to understand and change, and from which most other difficulties flow.	Introduction p.9
Invisibility	Software's lack of any natural visual or geometric form; because you cannot *see* its structure, you depend on diagrams, models, and visualization tools to reason about it.	Introduction p.9
Changeability	Software's constant exposure to change pressure because it is the most malleable part of any system; the reason maintenance dominates and version control exists.	Introduction p.9
Conformity	The obligation of software to match arbitrary external interfaces, standards, and institutions it cannot redesign; a major source of externally-imposed (not chosen) complexity.	Introduction p.9
Discontinuity	The tendency of small input or code changes to cause disproportionately large, non-continuous jumps in behaviour; why "small safe edits" cause regressions and why verification is mandatory.	Introduction p.9
Accidental properties	The replaceable, era-specific concerns — chosen technologies, operating systems, platforms — that turn over every few years; improving them cannot touch the essential difficulties.	Introduction p.10
Ad hoc paradigm	The earliest development mode: craft without a defined process; works for tiny one-person programs but does not scale to large or long-lived systems.	Introduction p.13–14
Waterfall	A linear, construction-style process that freezes each phase (including requirements) before the next begins; predictable in theory but broken by requirements volatility in practice.	Introduction p.15
Requirements volatility	The empirical finding that IT requirements change ~2–3%/month (1.0%–3.5% per month by software type); the anomaly that makes waterfall's frozen up-front spec obsolete before delivery.	Introduction p.16–17
CHAOS Reports	Standish Group's annual reports since 1994 (~50,000 projects), scoring success as *on time, on budget, satisfactory result*; the data showing high project failure rates that undermine waterfall.	Introduction p.17

Term	Definition	Source
Agile / iterative paradigm	A repeating Plan→Collaborate→Deliver loop (backlog → iteration → daily review → deliverable → feedback) that absorbs requirements change by working in small, re-planned increments.	Introduction p.17–18
Life-span model	A high-level description of the stages software passes through from conception to death; used to locate *where maintenance fits* in a product's whole existence.	Introduction p.20–21
Staged model	A single product's life as five stages — Initial development → Evolution → Servicing → Phase-out → Close-down — distinguishing deep change (Evolution) from patches (Servicing).	Introduction p.21
Evolution (stage)	The stage of substantial, valuable change (features, architecture) made while the team still understands the code; where refactoring and the change process belong.	Introduction p.21–22
Servicing/Maintenance (stage)	The degraded stage after evolution stops, where only small, cautious patches are affordable because understanding and/or code health has eroded.	Introduction p.21–22
Code decay	The gradual degradation (duplication, shortcuts, tangling) that pushes software from Evolution into Servicing; the same phenomenon later framed as technical debt.	Introduction p.22
Reengineering	The deliberate counter-force to decay — restoring code health (e.g. via refactoring) to move software Servicing → Evolution so real evolution can resume; the justification for this course.	Introduction p.22
Versioned staged model	The staged model run in parallel across multiple shipped versions, each living its own staged cycle; models real products that support several releases at once (current/LTS/legacy).	Introduction p.23
V-model	A model pairing each design level with a matching test level (requirements↔functional, system design↔system, unit design↔unit testing); makes testing a first-class, pre-planned activity, maintenance at the top.	Introduction p.24
Prototype model	A life span that builds an early throwaway prototype to elicit *corrected* requirements before real design: requirements → prototype → corrected requirements → design → implementation → maintenance.	Introduction p.25
Version Control System (VCS)	A tool that records a project's full change history and coordinates team changes — archiving versions, keeping history, enabling collaboration, enforcing discipline, allowing recovery.	Git Lab p.3
Add / Commit / Update	The three essential VCS actions: stage a change (Add), record it with a message (Commit), and fetch the latest shared state (Update); Git's commands are elaborations of these.	Git Lab p.4
Centralized VCS	An architecture with one central repository holding the authoritative history; clients have only a working copy and need the server for most operations (e.g. SVN).	Git Lab p.7
Decentralized /distributed VCS	An architecture where every clone is a complete repository with full local history, enabling offline commit/branch/diff and removing the single point of failure (e.g. Git).	Git Lab p.7, p.9
Git	The decentralized VCS created by Linus Torvalds to manage large projects (originally Linux) efficiently; makes `commit` local and `push` the act of sharing.	Git Lab p.9
Workspace (working dir)	Git area 1: your actual editable files on disk — the only area you change directly with an editor.	Git Lab p.11
Index / stage	Git area 2: the staging area holding exactly the changes selected for the next commit; lets you compose clean, deliberate commits.	Git Lab p.11
Local repository (HEAD)	Git area 3: your committed history stored locally (the decentralized part); HEAD points to your latest local commit.	Git Lab p.11

Term	Definition	Source
Remote repository (origin)	Git area 4: the shared server copy (e.g. on GitHub, named origin) that teammates push to and pull from — the team's synchronisation point.	Git Lab p.11, p.15
clone	The command that creates a complete local copy of a remote repository (all history) and links it to the remote as origin; how you join a project.	Git Lab p.11, p.15
commit vs push	`commit` records changes only to your *local* HEAD; a separate `push` publishes those commits to the remote — the defining behaviour of a decentralized VCS and a classic exam trap.	Git Lab p.11
pull	The combined command `fetch` + `merge`: download remote changes *and* integrate them into your workspace in one step.	Git Lab p.11, p.15
GitHub flow	A lightweight collaboration model where each member works on their own feature branch, keeping concurrent work isolated until it is pushed and integrated.	IntroLab p.1
Maven	The Java build system (3.8.x on JDK 11) used to compile, install and run the JHotDraw case study; `mvn clean install` builds it, `target/` (its output) is never committed.	IntroLab p.1
JHotDraw	A Java drawing-editor framework; the recurring course case study, a design-pattern teaching vehicle, and the target of all Clean Code refactoring in the portfolio.	Git Lab p.18; Introduction p.6
Paradigm (Kuhn)	The shared framework of assumptions, methods and exemplars a community works within; the History slide imports the term from Thomas Kuhn to describe ad hoc, waterfall and agile as successive software paradigms.	Introduction p.11
Anomaly (Kuhn)	An observation the prevailing paradigm cannot explain; accumulating anomalies force a paradigm shift — the deck casts requirements volatility and the CHAOS data as waterfall's anomalies.	Introduction p.11, p.16–17
John Wilder Tukey	Statistician dated 1958 on the History slide, associated with the first software applications and the birth of software as a distinct artifact, initially built by electrical engineers and mathematicians.	Introduction p.11
Fred Brooks / IBM OS/360	Manager of IBM's OS/360 project (slide dates him 1982); the History slide credits this experience with motivating software life-span models and waterfall-era process thinking.	Introduction p.11
Deployment (waterfall phase)	The fifth and final box of the waterfall diagram (Requirements → Design → Implementation → Testing → Deployment); notably the diagram contains no maintenance stage at all.	Introduction p.15
Tree-swing cartoon	Five-panel illustration of requirements distortion across role hand-offs: customer explained / project leader understood / analyst designed / programmer wrote / what the customer really wanted.	Introduction p.16
CHAOS Modern Resolution	The Standish success metric — on time, on budget, with a satisfactory result — under which only 27–31% of all projects succeeded in each year 2011–2015 (~70% challenged or failed).	Introduction p.17
Product life-cycle curve	The Introduction → Growth → Maturity → Decline sales curve borrowed from product management to motivate software life-span models.	Introduction p.20–21
fetch	Downloads remote commits into the local repository **without touching the workspace**; the first half of `pull`.	Git Lab p.11
merge	Integrates commits from the local repository into the workspace; the second half of `pull`.	Git Lab p.11
diff vs diff HEAD	`diff` compares workspace against the index (unstaged edits only); `diff HEAD` compares workspace against the last commit (all changes since HEAD) — after staging, `diff` is empty but `diff HEAD` is not.	Git Lab p.11
git status	Lists which files have changed in the workspace — the first inspection step before any commit.	Git Lab p.13

Term	Definition	Source
git diff [files]	Shows the line-level changes inside the named files — the second inspection step.	Git Lab p.13
git commit -a [-m \ <message>]	Records all tracked, modified files into local history in one step, with an optional inline message.	Git Lab p.13
git show	Displays the last commit, used to review what was just recorded.	Git Lab p.13
origin	The default name Git gives the remote repository a clone was created from; the team's synchronisation point on GitHub.	Git Lab p.15
Fork (GitHub)	A server-side copy of a repository under the team's own account, made so the team can push without write access to the original project.	IntroLab p.1; Git Lab p.18–19
Issue (GitHub)	GitHub's change-request/bug-tracker record; filing one is the lab's final TODO and the course's first instance of the Initiation phase.	Git Lab p.19
-DskipTests	Maven flag that skips test execution during the first smoke build of JHotDraw (`mvn clean install -DskipTests`).	IntroLab p.1
jhotdraw-samples-misc	The module folder from which the SVG sample GUI is launched via `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` .	IntroLab p.1
Linus Torvalds	Creator of Git (slide spelling "Linus Thorvalds"), who built it to manage large projects such as Linux efficiently.	Git Lab p.9
GIT Demo 1 / Demo 2	The lab's two live demonstrations: Demo 1 "Basics" covers the local-repository loop; Demo 2 "Advanced" covers remote repositories (clone/pull/push).	Git Lab p.12, p.14
Capers Jones table	Per-software-type monthly requirements-change rates: contract/outsource 1.0%, information systems 1.5%, system 2.0%, military 2.0%, commercial 3.5% (Jones, "Strategies for Managing Requirements Creep", IEEE Computer, 1996).	Introduction p.17

Common Pitfalls / Gotchas

- **Confusing essential vs accidental.** Essential difficulties (complexity, invisibility, changeability, conformity, discontinuity) cannot be engineered away; accidental properties (the technology/OS du jour) can. Exam answers that say "better tools remove complexity" are wrong — tools at best *manage* essential difficulties (Introduction p.9–10).
- **Confusing `commit` with `push`.** In Git, `commit` only records to your **local** repository (HEAD); your teammates see nothing until you `push`. This follows directly from Git being **decentralized** (every clone is a full local repository) and is a classic exam trap (Git Lab p.7, p.9, p.11).
- **`pull` is not atomic-magic.** `pull` = `fetch` + `merge`; if you only `fetch`, the remote changes are downloaded into your local repository but your *workspace* is unchanged until you `merge` (Git Lab p.11).
- **Committing the wrong things.** Do **not** commit executables or large binaries (e.g. a 2 GB file) or generated/local build output — they don't diff well, bloat history forever, and conflict badly; commit *source* only (Git Lab p.6). In the Maven/JHotDraw build, never commit `target/` (it is regenerated by `mvn clean install`).
- **Skipping the local inspection loop.** Always `git status` then `git diff` before committing, so you know exactly which files and which lines you're recording — this is what keeps stray edits and debug code out of history (Git Lab p.13).
- **Mixing up Evolution and Servicing.** Evolution = real, deep change while the team understands the code; Servicing = patches only, after evolution has stopped. **Code decay** drives Evolution→Servicing;

reengineering/refactoring is what reverses it — which is precisely *why* the course exists (Introduction p.21–22).

- **Thinking one paradigm "won".** Ad hoc, waterfall and agile **co-exist**; the exam expects you to know waterfall's anomalies (requirements volatility ~2–3%/month; CHAOS failure rates) *and* that waterfall still fits some stable-requirement, high-stakes contexts (Introduction p.16–19, p.27).
- **First build flags.** The IntroLab build deliberately uses `-DskipTests` for the smoke build (you only need it to compile first) and runs the GUI from `jhotdraw-samples-misc`; forgetting the exact main class `org.jhotdraw.samples.svg.Main` means the GUI won't launch (IntroLab p.1).
- **Brooks attribution.** The five essential difficulties come from Brooks's "No Silver Bullet", but the slides never name Brooks — don't claim the deck cites him (Grounding note). Precision matters here: Fred Brooks (1982) *is* named on the Software Engineering History slide, but only in connection with **IBM OS/360** and the rise of life-span models/waterfall (Introduction p.11) — the deck never connects him to the essential/accidental terminology of p.9–10.
- **Misreading the Capers Jones table.** The per-type monthly change rates are **1.0–3.5 percent per month** (contract/outsource 1.0%, information systems 1.5%, system 2.0%, military 2.0%, commercial 3.5%), *not* 10–35% — sanity-check against the same deck's headline "2 to 3 % per month" (Introduction p.16–17).
- **Two different CHAOS tables.** Introduction p.17 shows the *Modern Resolution for ALL projects by year* (2011–2015; successful never above 31%); Introduction p.19 shows *resolution segmented by method (agile vs waterfall) and size*. Quoting "agile 39% successful" from the wrong table, or "29% successful" as an agile number, is a recognisable error (Introduction p.17, p.19).
- **Slide typos — do not reproduce them.** The decks contain several spelling slips: "Comformity" (Introduction p.9), "conformatity" (Introduction p.27), "Addhoc" and "Tranined Skills" (Introduction p.11), "Casper-Jones" for Capers Jones (Introduction p.16), "Linus Thorvalds" for Linus Torvalds (Git Lab p.9), and "Syllabys" ([Litt] Literature List p.2). Exam answers should use the correct forms: *conformity*, *ad hoc*, *Capers Jones*, *Torvalds*.
- **Generic "Update" ≠ Git `fetch`.** The generic VCS action *Update* (Git Lab p.4–5) corresponds to Git's `pull` (= `fetch` + `merge`), not to `fetch` alone — after a bare `fetch` your workspace files are unchanged (Git Lab p.11).
- **`diff` after staging shows nothing.** `git diff` compares workspace against the *index*; once you `add -u`, the edits are in the index and plain `diff` is empty while `diff HEAD` still shows them — don't conclude your changes vanished (Git Lab p.11).
- **Waterfall has no maintenance box.** The waterfall diagram ends at **Deployment** (Requirements → Design → Implementation → Testing → Deployment); claiming the deck's waterfall includes a maintenance phase is wrong — its *absence* is the lecture's point (Introduction p.15).
- **V-model pairings must match levels.** Requirements ↔ functional testing, system design ↔ system testing, unit design ↔ unit testing, with implementation alone at the bottom and maintenance at the top — pairing unit testing with requirements (or any cross-level match) is a classic error (Introduction p.24).
- **Git ≠ GitHub.** Git is the decentralized VCS itself (Torvalds, runs entirely locally); GitHub is a hosting platform that adds forks, Issues and collaboration on top of remote Git repositories. The lab keeps them in separate sections (Git Lab p.8–15 vs p.16–19) — answers should too.
- **Reengineering is not a fifth life-span model.** The summary slide lists exactly four models (staged, versioned staged, V-model, prototype); reengineering is an overlay arrow (Servicing → Evolution) on the staged model, not a standalone model (Introduction p.22, p.28).

Exam Focus

High-probability exam material from Lecture 1:

- **The five essential difficulties** — be able to name and explain all five (complexity, invisibility, changeability, conformity, discontinuity), give the one-line "why it's hard" for each, and contrast them with accidental properties (the point being that essentials can be managed but not removed) (Introduction p.9–10, p.27).
- **The three paradigms and waterfall's failure** — define ad hoc, waterfall, agile; cite the two anomalies (requirements volatility ~2–3%/month; CHAOS success/challenged/failed rates) and the conclusion that paradigms co-exist rather than one replacing another (Introduction p.13–19, p.27).
- **Life-span models** — reproduce the **staged model** sequence (Initial development → Evolution → Servicing → Phase-out → Close-down) with its transitions, and explain **code decay** (Evolution→Servicing) vs **reengineering** (Servicing→Evolution) as opposing forces; know versioned-staged, V-model and prototype at a high level and where maintenance sits in each (Introduction p.21–25, p.28).
- **Why version control** — the six reasons, the Add/Commit/Update essential actions, and **centralized vs decentralized** (and why decentralization is what makes `commit` local) (Git Lab p.3–4, p.7).
- **The Git four-area model** — workspace → index/stage → local repo (HEAD) → remote, and which command moves between which areas, especially `commit` (local) vs `push` (remote) and `pull` = `fetch` + `merge` (Git Lab p.11).
- **Course logistics** — 5 ECTS, individual portfolio refactoring JHotDraw to Clean Code, assessed on the 7-point scale (Introduction p.4–6).
- **Case-study bootstrap** — fork on GitHub, Maven 3.8.x/JDK 11, `mvn clean install -DskipTests`, run the SVG `Main`, GitHub-flow feature branches (IntroLab p.1).
- **Forward link to the change process** — note that filing a GitHub Issue (Git Lab p.19) is the course's first **Initiation** step, and that reengineering/refactoring on JHotDraw is what the canonical change process operationalises in later lectures.
- **History names and dates** — John Wilder Tukey (1958, first software applications), Thomas Kuhn (paradigm → anomaly → shift), Fred Brooks (1982, IBM OS/360 → life-span models/waterfall); be able to reproduce the three threads of the History slide (Introduction p.11).
- **The two CHAOS tables, separately** — Modern Resolution for all projects by year (successful 27–31%, challenged 49–56%, failed 17–22% across 2011–2015; definition: on time, on budget, satisfactory result) versus the agile-vs-waterfall table segmented by size; know which numbers live in which table (Introduction p.17, p.19).
- **Capers Jones exact values and source** — contract/outsource 1.0%, information systems 1.5%, system 2.0%, military 2.0%, commercial 3.5% per month, from "Strategies for Managing Requirements Creep", C. Jones, 1996, IEEE Computer (Introduction p.17).
- **The tree-swing cartoon** — five panels in order (customer explained / project leader understood / analyst designed / programmer wrote / customer really wanted) as the qualitative half of the volatility anomaly (Introduction p.16).
- **Waterfall's exact five phases** — Requirements → Design → Implementation → Testing → Deployment, and the observation that maintenance is absent from the diagram (Introduction p.15).
- **Product life-cycle curve** — Introduction, Growth, Maturity, Decline and its mapping onto the staged model's stages (Introduction p.20–21).
- **The LAB TODOs** — the four tasks (clone from GitHub; learn the commands via the workflow slide; commit initial code with add/commit/pull/push; file a change request as a GitHub Issue) and why TODO

4 prefigures Initiation (Git Lab p.19).

- **Easily confused Git pairs** — `fetch` vs `pull`, `diff` vs `diff HEAD`, `clone` vs `pull`, generic Update vs Git `fetch`; each resolves by naming the two areas the command connects (Git Lab p.11).
- **Literature anchors** — [Raj13] Rajlich is the course textbook behind the paradigms/life-span material; [MC09] Clean Code is the standard the portfolio refactoring is graded against; [Sør15c] provides review questions specifically for this introduction lecture ([Litt] Literature List p.1–2).

Slide-by-Slide Source Walkthrough

A complete page-level companion to both decks, so that no slide is unaccounted for. Page references follow this guide's citation convention (see header). Section-divider slides carry only a title; they are listed because they reveal the decks' intended structure.

IntroductionSB5MAI.pdf — every slide

Slide	Title on slide	Content in brief
p.1	Introduction to Software Maintenance (SB5-MAI)	Title slide: course name, lecturer **Jan Corfixen Sørensen**, University of Southern Denmark.
p.2	Outline	The deck's parts: SB5-MAI: Software Maintenance · Introduction · Paradigms · Software Life-Span Models · Summary.
p.3	SB5-MAI: Software Maintenance	Section divider for the course-logistics block.
p.4	About SB5-MAI	Weight 5 ECTS / 0.083 FTE; prerequisites (advanced OO ≈12.5 ECTS; software engineering ≈20 ECTS; project team work); learning outcome: maintenance of larger existing software projects; teaching language English or Danish.
p.5	SB5-MAI Exam	Written report on campus; the individual portfolio constitutes the basis for examination; assessed on the 7-point grading scale.
p.6	SB5-MAI Portfolio	Refactor JHotDraw features according to Clean Code; **individual** mandatory portfolio examples.
p.7	Introduction	Section divider for the motivation/essential-difficulties block.
p.8–9	Motivation	Two-step build: "Internet in everything", then "Software is everywhere"; illustrated by the annotated Google self-driving car, a washing machine, and an AI head (Watson, Google Photos).
p.9	Essential Properties	The five essential difficulties with hand annotations: Complexity (polygons / all shapes / as needed), Invisibility (binary spiral + visualization-tools tree), Changeability (master/develop/topic branch diagram), Conformity ("Comformity"; M—d—D), Discontinuity (padlock + password).
p.10	Accidental Properties	Wall of technology/OS/platform logos annotated *Technologies*, *Operating systems*, *Roles of Software* (GWT, Joomla, Magento, WordPress, AJAX, Silverlight, .NET, PHP, Java, iPhone, Android, SharePoint, Facebook, Sitecore, …).
p.11	Software Engineering History	Three named threads: John Wilder Tukey (1958) → first SW applications → electrical engineers + mathematicians → ad hoc; Thomas Kuhn → paradigm → anomaly → complexity → ad hoc vs trained skills; Fred Brooks (1982) → IBM OS/360 → SW life-span models, waterfall.
p.12	Paradigms	Section divider for the paradigms block.

Slide	Title on slide	Content in brief
p.13–14	Ad hoc	Programmers recruited from the ranks of hardware engineers and mathematicians; software separated from hardware in the 1950s; software engineering defined as a field of its own.
p.15	Software Waterfall	Linear process; used in construction and manufacturing; intuitively appealing metaphor; diagram: Requirements → Design → Implementation → Testing → Deployment.
p.16	Anomaly of Requirements volatility	Tree-swing cartoon (five panels) + "Requirements for IT change at a rate 2 to 3 % per month (Casper-Jones)".
p.17	Standish Group Anomaly	CHAOS Reports yearly since 1994; 50,000 projects worldwide; new success definition (on time, on budget, satisfactory result); Modern Resolution table 2011–2015 (successful 29/27/31/28/29%, challenged 49/56/50/55/52%, failed 22/17/19/17/19%).
p.17–18	Agile Approach	Capers Jones per-type change-rate table (1.0–3.5%/month; Jones 1996, IEEE Computer) + the agile loop: Backlog (Items) → Iteration (Daily Review) → Release → Deliverable, with Feedback returning; captions Plan · Collaborate · Deliver.
p.19	Agile VS Waterfall	CHAOS resolution by agile vs waterfall, segmented by size (all/large/medium/small); footnote: FY2011–2015, new CHAOS database, total >10,000 projects.
p.20	Software Life-Span Models	Section divider for the life-span block.
p.20–21	Life-Span Models	Stages from conception to death; stages may be very different; similar to other products' life spans; product-sales curve over Introduction / Growth / Maturity / Decline.
p.21	Staged Model	Initial development —first version→ Evolution (loop: evolution changes) —evolution stops→ Servicing or Maintenance (loop: servicing patches) —servicing discontinued→ Phase-out —switch-off→ Close-down.
p.22	Reengineering	The staged model with force arrows: first running version; software changes (loop); **code decay** (Evolution→Servicing); **reengineering** (Servicing→Evolution); servicing patches (loop); servicing discontinued; switch-off.
p.23	Versioned Staged Model	Central evolution trunk spawning versions: Evolution/Servicing/Phase-out/Close-down for Version 1 and Version 2, then "Evolution Version …"; transitions: first running version, evolution changes, servicing patches, evolution of new version.
p.24	V-Model	Left arm: requirements → system design → unit design → implementation; right arm: unit testing → system testing → functional testing → maintenance; dashed pairings across the V.
p.25	Prototype Model	requirements → prototype → corrected requirements → design → implementation → maintenance.
p.26	Summary	Section divider for the summary block.
p.27	Software Engineering (summary)	Software a product since the 1950s; essential difficulties (complexity, invisibility, changeability, "conformatity" [sic], discontinuity); three paradigms (ad hoc, waterfall, iterative); the paradigms co-exist.
p.28	Software Life-Span Models (summary)	Life-span models offer a simplified and comprehensive view of the entire software engineering discipline; four models listed: staged, versioned staged, V-model, prototype.

Lab - GIT.pdf — every page

Page	Title on slide	Content in brief
p. 1	Version Control — Lab	Title slide of the lab deck.
p. 2	Version Control	Opening picture: four developers exchanging documents bidirectionally with a central database cylinder labelled "(VCS)".
p. 3	Why Version Control Systems?	Six illustrated reasons: enforce discipline; archive versions; maintain historical information; enable collaboration; recover from accidental deletions or edits; conserve disk space.
p. 4	Essential Actions	The three irreducible VCS actions: Add (+ icon), Commit (up arrow), Update (down arrow).
p. 5	VCS Workflow	Two developers and a central repository; left developer add+commits `foo.txt` and updates to receive `foo.txt'`; right developer updates to receive `foo.txt` and commits `foo.txt'`.
p. 6	Don't do it	Crossed-out executable (.EXE) and 2 GB file; repository should receive the VCS (source) file, not the local version (generated) file.
p. 7	Types of VCS	Centralized: developers ↔ one central repository. Decentralized: each developer ↔ own small local repository ↔ multiple peer repositories.
p. 8	GIT — Lab	Section divider: from generic VCS to Git specifically.
p. 9	About GIT	Decentralized VCS; Linus Thorvalds [sic — Torvalds]; large projects in an efficient way; for example Linux.
p. 10	Install GIT	Linux: package; Mac OS: XCode, package; Windows: package with installer; see `http://git-scm.com/downloads`.
p. 11	Git Workflow	The four areas — workspace (working dir), index (stage), local repository (HEAD), remote repository — with arrows clone, add(-u), commit, push, fetch, merge, pull, diff HEAD, diff.
p. 12	GIT Demo 1 — Basics	Divider for the first live demo (local-repository basics).
p. 13	Local Repositories	"A possible workflow (this is just an example!)": some editing → `git status` (see what files you changed) → `git diff [files]` (see the changes) → `git commit -a [-m <message>]`; inspection commands `git status`, `git diff`, `git show (last commit)`; left margin shows the partially cut-off create sequence (mkdir / cd / git init…).
p. 14	GIT Demo 2 — Advanced	Divider for the second live demo (remote repositories).
p. 15	Remote Repositories	Copy repository: `git clone <repository>` (repository = URL: file, http, ssh, …; creates a complete local copy; links it to the remote repository (origin); you can also link to a repository later). Receive changes: `git pull`. Send changes: `git push`.
p. 16	GIT Hub	Section divider: from Git the tool to GitHub the hosting platform.

Page	Title on slide	Content in brief
p. 17	LAB	Section divider: the lab assignment itself.
p. 18	Projects	The course project: **JHotDraw**.
p. 19	LAB TODOs	Clone remote repository from GitHub; get familiar with Git commands (see the workflow slide for an overview); commit your initial project code using add, commit, pull, push…; input change request as an Issue at GitHub.

IntroLab.pdf and the Literature List — page coverage

- **IntroLab p.1** — the entire one-page handout: introduction ("check-out our CASE study source code"), objectives (getting started with Maven; getting started with GitHub workflow), and classwork (team forks `[JHotDraw]` on GitHub; members follow `[GitHub flow]`, each on their own feature branch; install `[Maven build system]` 3.8.x based on JDK11; `mvn clean install -DskipTests` at project root; `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` at `jhotdraw-samples-misc` folder level; the JHotDraw GUI should then be showing).
- **[Litt] Literature List p.1–2** — the full course bibliography, headed "Software Maintenance (SB-MAI) — Literature", 29 entries from [BBPR05] to [VRvdL+16]; transcribed in full in the annotated bibliography section below.

Course Literature — Complete Annotated Bibliography

The literature list ([Litt] Literature List p.1–2) is the course's reading map; the bracketed keys are how every later lecture cites its sources. All 29 entries, grouped by the course thread they support:

Course backbone

- **[Raj13]** Vaclav Rajlich. *Software Engineering: The Current Practice*. ACM, New York, 2013 (Book). — **The course textbook.** This introduction lecture's paradigms, anomalies and life-span models track its opening chapters; the canonical change process of later lectures is its core.
- **[MC09]** Robert C. Martin and James O. Coplien. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall, 2009. — **The grading standard**: the portfolio task is to refactor JHotDraw features *according to Clean Code* (Introduction p.6).
- **[Sør15a]** Jan Sørensen. *Introduction to Software Maintenance*, 2015 (Video). — Companion video to this very lecture.
- **[Sør15b]** Jan Sørensen. *Overview of Software Maintenance Syllabys* [sic], 2015 (Video). — Course-overview video.
- **[Sør15c]** Jan Sørensen. *Review questions - Introduction to Software Maintenance*, 2015 (HTML). — **Review questions exist specifically for this lecture**; use them as a self-test source.

JHotDraw case-study cluster

- **[Kai01]** Wolfram Kaiser. *Become a programming Picasso with JHotDraw*, 2001 (HTML). — Hands-on tutorial.
- **[Kir01]** Douglas Kirk. *JHotDraw Pattern Language*, 2001 (HTML). — The framework's design patterns.

- **[Ran11a]** Werner Randelshofer. *JHotDraw 7 Documentation*, 2011 (HTML). — Reference documentation.
- **[Ran11b]** Werner Randelshofer. *The JHotDraw 7 Handbook*, 2011 (PDF). — The handbook for JHotDraw 7.
- **[Pav11]** Nikolaidis Pavlos. *Software Requirements Specification for JHotDraw*, 2011 (PDF). — An SRS for the case study.
- **[Sav01]** Jolita Savolskyte. *Review of the JHotDraw framework*, 2001 (PDF). — External review of the framework.

Refactoring and maintainability cluster

- **[Fow11]** Martin Fowler. *Refactorings in Alphabetical Order*, 2011 (HTML). — The refactoring catalog used when naming individual refactorings.
- **[Ker05]** Joshua Kerievsky. *Refactoring to patterns*. Addison-Wesley, 2005 (Book). — Bridges refactoring and design patterns.
- **[Sou]** SourceMaking.com. *Refactorings* (HTML). — Online refactoring reference.
- **[VRvdL+16]** Joost Visser, Sylvan Rigal, Rob van der Leek, Pascal van Eck, Gijs Wijnholds. *Building Maintainable Software, Java Edition: Ten Guidelines for Future-Proof Code*. O'Reilly, 1st ed., 2016 (Book). — Ten concrete maintainability guidelines for Java.
- **[GHJV94]** Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. — The Gang-of-Four patterns book; JHotDraw is famously built from these patterns.

Testing and verification cluster

- **[JUn]** JUnit. *JUnit Cookbook* (HTML). — The unit-testing framework used in the labs.
- **[Cos16]** Joel Costigliola. *AssertJ*, 2016 (HTML). — Fluent assertions library used alongside JUnit.
- **[Sch16]** Jan Schäfer. *JGiven*, 2016 (HTML). — BDD-style given/when/then testing for Java.
- **[Nor06]** Dan North. *Introducing BDD*, 2006 (HTML). — The original behaviour-driven development essay.
- **[Mes03]** Gerard Meszaros. *The Test Automation Manifesto*, 2003 (PDF). — Principles of automated testing.
- **[Mes08]** Gerard Meszaros. *xUnit Patterns*, 2008 (HTML). — Patterns for structuring xUnit-family tests.

Change-process and comprehension-tools cluster

- **[RG04]** V. Rajlich and P. Gosavi. *Incremental Change in Object-oriented Programming*. IEEE Software, 2004 (PDF). — The incremental-change process paper behind the course's change workflow.
- **[BMZ+05]** Jim Buckley, Tom Mens, Matthias Zenger, Awais Rashid, Günter Kniesel. *Towards a taxonomy of software change*. Journal of Software Maintenance and Evolution, 17(5):309–332, September 2005 (PDF). — How kinds of software change are classified.
- **[BBPR05]** Jonathan Buckner, Joseph Buchta, Maksym Petrenko, Václav Rajlich. *JRipples: A Tool for Program Comprehension during Incremental Change*. IWPC, pages 1–4, 2005 (PDF). — The **JRipples** tool for impact analysis / program comprehension during incremental change.
- **[Ols12a]** Andrzej Olszak. *Featureous: an integrated approach to location, analysis and modularization of features in java applications*. PhD thesis, 2012 (PDF). — The **Featureous** feature-location tool.
- **[Ols12b]** Andrzej Olszak. *Introduction to Featureous*, 2012 (Video). — Companion video for Featureous.

Process and version-control support

- **[Fow06]** Martin Fowler. *Continuous Integration*, 2006 (HTML). — The CI essay backing the course's Software Processes / CI topic.
- **[Lea16]** LearnCode. *Github Tutorial*, 2016 (Video). — Video support for exactly this Git/GitHub lab.

Why this matters. The toolchain the course brief names — JHotDraw, Featureous, JRipples, GitHub, AssertJ, JUnit, JGiven — is anchored in this list ([Kai01]/[Ran11a/b], [Ols12a/b], [BBPR05], [Lea16], [Cos16], [JUn], [Sch16] respectively); only Maven arrives via the IntroLab handout instead (IntroLab p.1). Note also the list's header spells the course code "**SB-MAI**" while the lecture deck uses "SB5-MAI" — same course. Topic tags: **Version Control / Git**, **Software Testing**, **Refactoring**, **JHotDraw Case Study**.

Compare & Contrast Quick-Reference Tables

Side-by-side distinctions for the concepts this lecture most often asks you to tell apart.

The three paradigms

	Ad hoc	**Waterfall**	**Agile / Iterative**
Era / origin	1950s; programmers recruited from hardware engineers and mathematicians (Introduction p.13–14)	Borrowed from construction and manufacturing (Introduction p.15)	Response to waterfall's anomalies (Introduction p.16–18)
Process shape	None — individual craft	Linear, one pass: Requirements → Design → Implementation → Testing → Deployment	Repeating loop: Plan → Collaborate → Deliver (Backlog → Iteration → Release, Feedback returns)
Requirements assumption	Implicit, in one person's head	Fully fixable up front, then frozen	Expected to change ~1.0–3.5%/month; re-planned every iteration
Strength	Cheap for tiny, short-lived programs	Predictable *if* requirements are stable; front-loaded planning	Absorbs change; best CHAOS success rates at every size (Introduction p.19)
Failure mode	Does not scale to teams or longevity	Requirements volatility + CHAOS failure rates (Introduction p.16–17)	(Not given on the slides — the deck's data favour agile throughout)
Status today	Co-exists	Co-exists	Co-exists (Introduction p.27)

The life-span models

Model	**Shape**	**Key stages / pairings**	**Where maintenance sits**	**Distinctive idea**
Staged (Introduction p.21)	Single descending chain with self-loops	Initial development → Evolution → Servicing/Maintenance → Phase-out → Close-down	The Servicing stage (and Evolution before it)	Evolution vs Servicing distinction

Model	Shape	Key stages / pairings	Where maintenance sits	Distinctive idea
Reengineering overlay (Introduction p.22)	Staged model + opposing arrows	code decay: Evolution→Servicing; reengineering: Servicing→Evolution	Same as staged	Decay is reversible — at a cost
Versioned staged (Introduction p.23)	Trunk spawning parallel staged lives	Evolution Version 1, 2, … each with its own Servicing/Phase-out/Close-down	Per version, concurrently	Multiple supported releases at once
V-model (Introduction p.24)	A "V" of paired levels	requirements↔functional testing; system design↔system testing; unit design↔unit testing; implementation at the bottom	Top of the right arm, above functional testing	Every design level pre-paired with a test level
Prototype (Introduction p.25)	Linear with an early learning loop	requirements → prototype → corrected requirements → design → implementation → maintenance	Terminal stage	Buy understanding early with a throwaway build

Centralized vs decentralized VCS at a glance

	Centralized (e.g. CVS, SVN)	Decentralized (Git)
Where full history lives	Only on the central server	In every clone (Git Lab p.7)
Offline work	Mostly blocked — server needed for commit/history/branch	Full: commit, history, branch, diff all local (Git Lab p.9, p.11)
Single point of failure	Yes — the server	No — any clone can restore the project
Sharing a change	`commit` reaches everyone directly	`commit` is local; `push` publishes (Git Lab p.11)
Course relevance	The simpler model taught first (Git Lab p.5)	What Git/GitHub actually is (Git Lab p.9–15)

Generic VCS actions vs Git commands

Generic action (Git Lab p.4)	Git realisation (Git Lab p.11)	Note
Add	`git add` / `git add -u`	`-u` stages tracked files' modifications only
Commit	`git commit` **plus** `git push`	Decentralization splits "record" from "publish"
Update	`git pull` = `fetch` + `merge`	A bare `fetch` is *not* an update of your workspace

Easily confused Git pairs at a glance

Pair	The one-line distinction (all Git Lab p.11)
`commit` vs `push`	Record locally vs publish to the remote
`fetch` vs `pull`	Download only vs download **and** merge into workspace
`diff` vs `diff HEAD`	Against the index (unstaged only) vs against the last commit (everything)
`clone` vs `pull`	Join a project (full copy, once) vs synchronise an existing clone (continually)
workspace vs index	Files you edit vs changes selected for the next commit

Evolution vs Servicing (staged model)

	Evolution	**Servicing / Maintenance**
Kind of change	Substantial evolution changes: features, architecture (Introduction p.21)	Small servicing patches only
Precondition	Team still understands the code; design healthy	Understanding/health eroded
Driving transition	Entered via *first (running) version*	Entered via *evolution stops* / **code decay** (Introduction p.22)
Way back	—	**Reengineering** returns Servicing → Evolution
Course connection	Where refactoring and the change process live; the portfolio works here	The fate the course teaches you to avoid/reverse

Essential vs accidental properties

	Essential (Introduction p.9)	**Accidental (Introduction p.10)**
What they are	Complexity, invisibility, changeability, conformity, discontinuity	Technologies, operating systems, roles of software (the logo wall)
Lifetime	Permanent — intrinsic to software as such	Era-specific — many slide logos are already obsolete
Can tools remove them?	No — only manage them	Yes — they are replaceable choices
Exam claim	"No tool removes complexity" is the expected answer	"A new framework solves it" is the trap answer

Self-Test Questions

Answers are grounded in the cited slides; attempt before reading the answer line.

1. **Q:** What is SB5-MAI's weight, learning outcome and assessment form? **A:** 5 ECTS (0.083 FTE); maintenance of larger existing software projects; written report on campus based on the individual portfolio, graded on the 7-point scale (Introduction p.4–5).
2. **Q:** What exactly is the portfolio task? **A:** Refactor JHotDraw features according to Clean Code, as individual mandatory portfolio examples (Introduction p.6).
3. **Q:** Name the five essential difficulties and the canonical illustration of each. **A:** Complexity (polygon tessellation), invisibility (binary spiral + visualization tools), changeability (master/develop/topic branches), conformity (M—d—D constraint), discontinuity (padlock + password) (Introduction p.9).
4. **Q:** Why can no new framework remove the essential difficulties? **A:** Frameworks are accidental properties — replaceable, era-specific choices; essential difficulties are intrinsic to software itself and can only be managed (Introduction p.9–10).
5. **Q:** Which three people are named on the History slide, and for what? **A:** John Wilder Tukey (1958, first software applications, staffed by electrical engineers and mathematicians), Thomas Kuhn (paradigm → anomaly), Fred Brooks (1982, IBM OS/360 → life-span models/waterfall) (Introduction p.11).
6. **Q:** Reproduce waterfall's five phases. What is missing from the diagram? **A:** Requirements → Design → Implementation → Testing → Deployment; there is no maintenance stage at all (Introduction p.15).
7. **Q:** State both anomalies of the waterfall paradigm with numbers. **A:** Requirements volatility — 2–3%/month overall (Capers Jones), 1.0–3.5%/month by software type; CHAOS data — only 27–31% of all

projects successful in any year 2011–2015 under the on-time/on-budget/satisfactory definition (Introduction p.16–17).

8. **Q:** In the CHAOS agile-vs-waterfall table, what are the success rates for large projects? **A:** Agile 18%, waterfall 3% — the largest relative gap of any size class (Introduction p.19).
9. **Q:** Name the agile loop's elements as drawn. **A:** Backlog (Items) feeds an Iteration with a Daily Review, producing a Deliverable at each Release, with Feedback flowing back; captions Plan, Collaborate, Deliver (Introduction p.17–18).
10. **Q:** Did agile replace waterfall? **A:** No — the deck's summary states the three paradigms (ad hoc, waterfall, iterative) co-exist (Introduction p.27).
11. **Q:** List the staged model's five stages and the named transitions. **A:** Initial development —first version→ Evolution —evolution stops→ Servicing/Maintenance —servicing discontinued→ Phase-out —switch-off→ Close-down, with loops *evolution changes* and *servicing patches* (Introduction p.21).
12. **Q:** What pushes software from Evolution to Servicing, and what pulls it back? **A:** Code decay pushes it down; reengineering (e.g. refactoring) pulls it back up (Introduction p.22).
13. **Q:** How does the versioned staged model differ from the staged model? **A:** A central evolution trunk spawns multiple versions, each living its own staged life concurrently — evolution of a new version begins before the old version closes down (Introduction p.23).
14. **Q:** Give the V-model's three pairings and the two unpaired boxes. **A:** requirements↔functional testing, system design↔system testing, unit design↔unit testing; implementation sits alone at the bottom and maintenance at the top of the right arm (Introduction p.24).
15. **Q:** What is the prototype model's sequence and purpose? **A:** requirements → prototype → corrected requirements → design → implementation → maintenance; an early throwaway build elicits corrected requirements before real design (Introduction p.25).
16. **Q:** Which four life-span models does the deck's own summary list? **A:** Staged, versioned staged, V-model, prototype — reengineering is an overlay, not a listed model (Introduction p.28).
17. **Q:** Give all six reasons for version control. **A:** Enforce discipline; archive versions; maintain historical information; enable collaboration; recover from accidental deletions or edits; conserve disk space (Git Lab p.3).
18. **Q:** What are the three essential VCS actions, and Git's realisation of each? **A:** Add (`git add` / `add -u`), Commit (`git commit` + `git push` to share), Update (`git pull` = `fetch` + `merge`) (Git Lab p.4, p.11).
19. **Q:** What must never be committed, and why? **A:** Executables and large binaries (the .EXE / 2 GB illustration) and generated/local build output — they don't delta well, bloat history, and are reproducible from source; in the course, Maven's `target/` (Git Lab p.6).
20. **Q:** Why does Git make `commit` local and `push` separate? **A:** Because Git is decentralized — every clone is a complete repository with full history, so recording (local) and publishing (remote) are different operations (Git Lab p.7, p.9, p.11).
21. **Q:** Your teammate says "I fetched but my files didn't change." Explain. **A:** `fetch` only downloads commits into the local repository; the workspace changes only after `merge` (or use `pull`, which does both) (Git Lab p.11).
22. **Q:** After `git add -u`, why does `git diff` show nothing? **A:** Plain `diff` compares workspace against the index; staging moved the edits into the index. `git diff HEAD` still shows them, comparing against the last commit (Git Lab p.11).
23. **Q:** What are the four LAB TODOs? **A:** Clone the remote repository from GitHub; get familiar with the Git commands (via the workflow slide); commit initial project code using add, commit, pull, push; input a change request as an Issue at GitHub (Git Lab p.19).
24. **Q:** Reproduce the JHotDraw bootstrap commands. **A:** Fork `[JHotDraw]` on GitHub; install Maven 3.8.x on JDK 11; `mvn clean install -DskipTests` at the project root; `mvn exec:java "-`

`Dexec.mainClass=org.jhotdraw.samples.svg.Main"` from `jhotdraw-samples-misc`; the GUI should appear (IntroLab p.1).

25. **Q:** Which literature keys back (a) the course textbook, (b) the refactoring grading standard, (c) the JRipples tool, (d) the Featureous tool, (e) BDD? **A:** (a) [Raj13] Rajlich; (b) [MC09] Clean Code; (c) [BBPR05]; (d) [Ols12a]/[Ols12b]; (e) [Nor06] (with [Sch16] JGiven as the Java tool) ([Litt] Literature List p.1–2).

Source Map

Deck / Lab	Pages	Sections of this guide that drew from them
`IntroductionSB 5MAI.pdf`	p.2 (outline)	Overview, Learning Objectives
`IntroductionSB 5MAI.pdf`	p.4	Course context; Definitions
`IntroductionSB 5MAI.pdf`	p.5–6	Assessment (portfolio, 7-point scale); JHotDraw Connection
`IntroductionSB 5MAI.pdf`	p.8–9	Motivation; Five essential difficulties
`IntroductionSB 5MAI.pdf`	p.10	Accidental properties
`IntroductionSB 5MAI.pdf`	p.11, p.13–14	Software engineering history; Ad hoc paradigm
`IntroductionSB 5MAI.pdf`	p.15–16	Waterfall; requirements volatility
`IntroductionSB 5MAI.pdf`	p.17	Agile cycle; per-type change-rate table; CHAOS/Standish
`IntroductionSB 5MAI.pdf`	p.18–19	Agile approach; Agile-vs-Waterfall CHAOS resolution table
`IntroductionSB 5MAI.pdf`	p.20–21	Life-span models overview; Staged model
`IntroductionSB 5MAI.pdf`	p.22	Reengineering / code decay
`IntroductionSB 5MAI.pdf`	p.23	Versioned staged model
`IntroductionSB 5MAI.pdf`	p.24	V-model
`IntroductionSB 5MAI.pdf`	p.25	Prototype model
`IntroductionSB 5MAI.pdf`	p.27–28	Summary (essential difficulties, paradigms, life-span models) — Exam Focus, Pitfalls
`Lab - GIT.pdf`	p.3	Why version control (six reasons)
`Lab - GIT.pdf`	p.4	Essential VCS actions (Add/Commit/Update)
`Lab - GIT.pdf`	p.5	VCS central-repository workflow

Deck / Lab	Pages	Sections of this guide that drew from them
`Lab – GIT.pdf`	p.6	"Don't do it" — what not to commit
`Lab – GIT.pdf`	p.7	Centralized vs decentralized VCS
`Lab – GIT.pdf`	p.9–10	About Git (Torvalds/Linux); installation
`Lab – GIT.pdf`	p.11	Git four-area model & command map
`Lab – GIT.pdf`	p.13	Local repository workflow
`Lab – GIT.pdf`	p.15	Remote repository workflow (clone/pull/push)
`Lab – GIT.pdf`	p.16, p.18–19	GitHub; Projects = JHotDraw; LAB TODOs
`IntroLab.pdf`	p.1	JHotDraw fork, Maven build/run, GitHub flow — Worked Example, JHotDraw Connection
`[Litt] Literature List.pdf`	p.1–2	Citation keys ([Raj13], [GHJV94], [MC09], [Kai01], [Kir01], [Ran11a/b], …) referenced in headers
`IntroductionSB 5MAI.pdf`	p.1–3, p.7, p.12, p.20 region, p.26 region (title & section dividers)	Deck roadmap; Slide-by-Slide Source Walkthrough
`IntroductionSB 5MAI.pdf`	p.11 (Tukey, Kuhn, Brooks threads)	Software Engineering History — Tukey, Kuhn, and Brooks by name
`IntroductionSB 5MAI.pdf`	p.15–17 (phase boxes, tree-swing, CHAOS Modern Resolution, Capers Jones values)	Waterfall's exact phase sequence; Tree-swing cartoon; CHAOS Modern Resolution table; Capers Jones table
`IntroductionSB 5MAI.pdf`	p.20–21 (product-sales curve)	Product life-cycle curve subsection
`IntroductionSB 5MAI.pdf`	p.27–28 (summary wording, four-model list)	What the Summary slides single out
`Lab – GIT.pdf`	p.1–2 (title; opening "(VCS)" picture)	Version Control Lab deck structure subsection; Slide-by-Slide Source Walkthrough
`Lab – GIT.pdf`	p.8, p.12, p.14, p.16–17 (section dividers; Demo 1 Basics; Demo 2 Advanced)	Version Control Lab deck structure subsection; Slide-by-Slide Source Walkthrough
`Lab – GIT.pdf`	p.11, p.13, p.15 (command arrows read closely)	Git command semantics in depth; The two-developer foo.txt scenario
`Lab – GIT.pdf`	p.19 (four TODO bullets verbatim)	LAB TODOs — the four tasks verbatim
`[Litt] Literature List.pdf`	p.1–2 (all 29 entries)	Course Literature — Complete Annotated Bibliography; JHotDraw in the course literature